Rockchip HDMI Developer Guide

ID: RK-SM-YF-119

Release Version: V1.2.0

Release Date: 2024-03-29

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

This document mainly introduces the usage and debugging methods of HDMI based on the DRM display framework on the Rockchip platform.

Overview

Product Version

Chip Models	Kernel Version
RK322X/RK3328/RK3368/RK3399/ RK3288/RK3528/RK356X/RK3576/RK3588	Linux kernel 6.1/5.10/4.19/4.4

Target Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Date	Version	Author	Modification Description
2020-06-24	V1.0.0	Cao Ruijie	Initial release
2020-08-26	V1.1.0	Cao Ruijie	Added HDCP 2.2 usage instructions Added RK3288/RK3399 HDMI-PHY-PLL modification method
2024-03-29	V1.2.0	Cao Ruijie	Removed FB framework introduction, made into a new document Added RK3528/RK3576/RK3588 instructions Added DTS configuration INNO HDMI PHY signal strength instructions Fixed color format description error Updated HDMI forced output resolution method

Rockchip HDMI Developer Guide
1. Rockchip Platform HDMI Overview
2. DRM Framework HDMI Introduction
2.1 HDMI Software Feature Configuration
2.1.1 Enabling HDMI
2.1.2 Binding VOP
2.1.3 Enabling Boot Logo
2.1.4 Binding VOP dclk to PLL
2.1.4.1 RK3288 PLL Binding
2.1.4.2 RK3399 Binding PLL
2.1.4.3 RK356X Binding PLL
2.1.4.4 RK3588/RK3576 Binding PLL
2.1.5 Enabling HDCP for RK3288/RK3399/RK3528/RK356X
2.1.5.1 Enable HDCP 1.4
2.1.5.2 Enabling HDCP 2.2
2.1.6 Enabling HDCP on RK3588/RK3576
2.1.7 DDC I2C Speed Configuration
2.1.8 HDMI Signal Strength Configuration
2.1.8.1 Signal Strength Configuration for RK322X/RK3328/RK3528/RK356X
2.1.8.1.1 RK322X Signal Configuration
2.1.8.1.2 RK3328 Signal Configuration
2.1.8.1.3 RK3528 Signal Configuration
2.1.8.2 RK3288/RK3368/RK3399/RK356X Signal Configuration
2.1.9 Added Special Resolutions
2.1.9.1 Adding Special Resolution Timings
2.1.9.2 New PLL Configuration for RK322X/RK3328/RK3528
2.1.9.2.1 PRE-PLL Calculation Process
2.1.9.2.2 POST-PLL Calculation Process
2.1.9.2.3 Using the Calculation Tool
2.1.9.3 New PLL Configuration for RK3288/RK3368/RK3399/RK356X
2.1.9.4 New PLL Configuration for RK3588/RK3576
2.1.10 Open Audio
2.2 Android Display Framework Configuration
2.2.1 Primary and Secondary Display Interface Configuration
2.2.2 Main and Auxiliary Display Interface Query
2.2.3 Framebuffer Resolution Configuration
2.2.4 Resolution Filtering Configuration
2.2.5 HDMI Configuration Options
2.3 Common Debugging Methods
2.3.1 Checking VOP Status
2.3.2 Checking Connector Status
2.3.3 Check HDMI Operation Status
2.3.4 Checking HDMI CEC Status
2.3.5 Force Enable/Disable HDMI
2.3.6 Command Line Resolution Configuration
2.3.6.1 Resolution Settings for Android 7.x & Android 8.x
2.3.6.2 Resolution Settings for Android 9.0 and Above
2.3.7 Setting Colors via Command Line
2.3.7.1 Android 7.x & Android 8.x Color Settings
2.3.7.2 Color Settings for Android 9.0 and Later versions
2.3.8 Setting Overscan
2.3.8.1 Android 7.x & Android 8.x Overscan Settings

2.3.8.2 Overscan Settings for Android 9.0 and Later versions
2.3.9 Setting Brightness, Contrast, Saturation, and Hue
2.3.9.1 Android 7.x & Android 8.x Brightness, Contrast, Saturation, and Hue Settings
2.3.9.2 Brightness, Contrast, Saturation, and Hue Settings for Android 9.0 and Above
2.4 Troubleshooting Common Issues
2.4.1 Inserting or switching resolutions, TV prompts no signal or format not supported or unstable image, intermittent, or with a large number of colored bright spots, bright lines
2.4.2 Video Playback Results in TV Signal Loss or Unsupported Format Notification
2.4.3 Some TVs show no signal, black screen, or garbled screen
2.4.4 How to Set Default Resolution When Reading EDID Fails
2.4.5 Force Output of Specified Resolution
2.4.6 Recovery HDMI Display Not Available
2.4.7 Unable to Set HDMI Resolution in settings
2.4.8 DDR Bandwidth Insufficiency Issues
2.4.9 4K UI Related Issues
2.4.10 Setting HDMI Resolution List Missing 4K Resolution
2.4.11 RK3588 Configuration: HDMI Resolution List Missing 8K Resolution
2.4.12 RK3588/RK3576 HDMI 8K/4K120 and Other Resolutions Screen Flicker, Display Anomalies
2.4.13 HDMI Certification Application Form Filling

1. Rockchip Platform HDMI Overview

The HDMI features of various Rockchip platforms are as follows:

Function	RK3288	RK3368	RK322X	RK3328	RK3399	RK3528	RK356X	RK3588	RK3576
Maximum Output Resolution	3840x2160p60	4096x2160p60	4096x2160p60	4096x2160p60	4096x2160p60	4096x2160p60	4096x2160p60	7680x4320p60	4096x2160p120
Interlaced Mode	N	N	Y	Y	Y	Y	Y	Y	Y
4K-60/50 Hz Supported Colors	RGB YCbCr444 YCbCr422 YCbCr420 (only RK3288W supports)	YCbCr420	YCbCr420	YCbCr420	RGB YCbCr444 YCbCr422 YCbCr420	RGB YCbCr444 YCbCr422 YCbCr420	RGB YCbCr444 YCbCr422 YCbCr420	RGB YCbCr444 YCbCr420	RGB YCbCr444 YCbCr422 YCbCr420
Support for 10-bit Color Depth	Y	N	Y	Y	Y	Y	Y	Y	Y
Supported HDMI Protocol Version	HDMI 2.0	HDMI 2.0	HDMI 2.0	HDMI 2.0	HDMI 2.0	HDMI 2.0	HDMI 2.0	HDMI 2.1	HDMI 2.1

- DRM:

  DRM stands for Direct Rendering Manager, which is a component of the DRI (Direct Rendering Infrastructure) framework. Kernels starting from Linux 4.4 use the DRM framework, and the HDMI driver path is:

  ```
  kernel/drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c
  kernel/drivers/gpu/drm/rockchip/inno_hdmi.c
  kernel/drivers/gpu/drm/bridge/synopsys/
  ```

2. DRM Framework HDMI Introduction

2.1 HDMI Software Feature Configuration

2.1.1 Enabling HDMI

To enable HDMI, add the following:

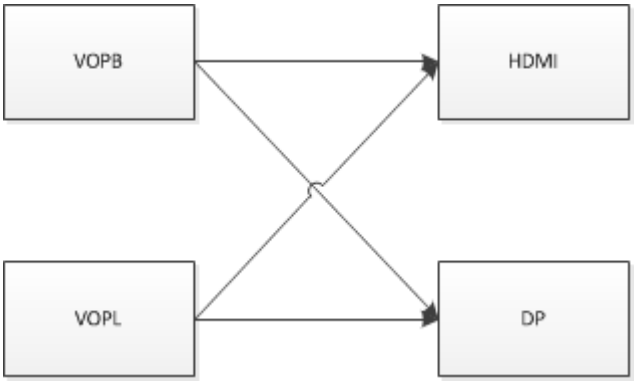
```
&hdmi {
    status = "okay";
};
```

2.1.2 Binding VOP

In various platforms of Rockchip, image data output from various display interfaces (HDMI, DP, CVBS, etc.) comes from VOP:



If the platform has two VOPs (RK3288, RK3399): VOPB (supports 4K), VOPL (only supports 2K), the two VOPs can be bound to two display interfaces respectively (one display interface can only be bound to one VOP), and can be interchanged with each other:



When the display device node is enabled in dts, the ports corresponding to VOPB and VOPL for the display interface will both be enabled, so it is necessary to disable the port of the VOP that is not needed. For example, if HDMI is bound to VOPB, add:

```
&hdmi_in_vopl {
    status = "disabled";
};
```

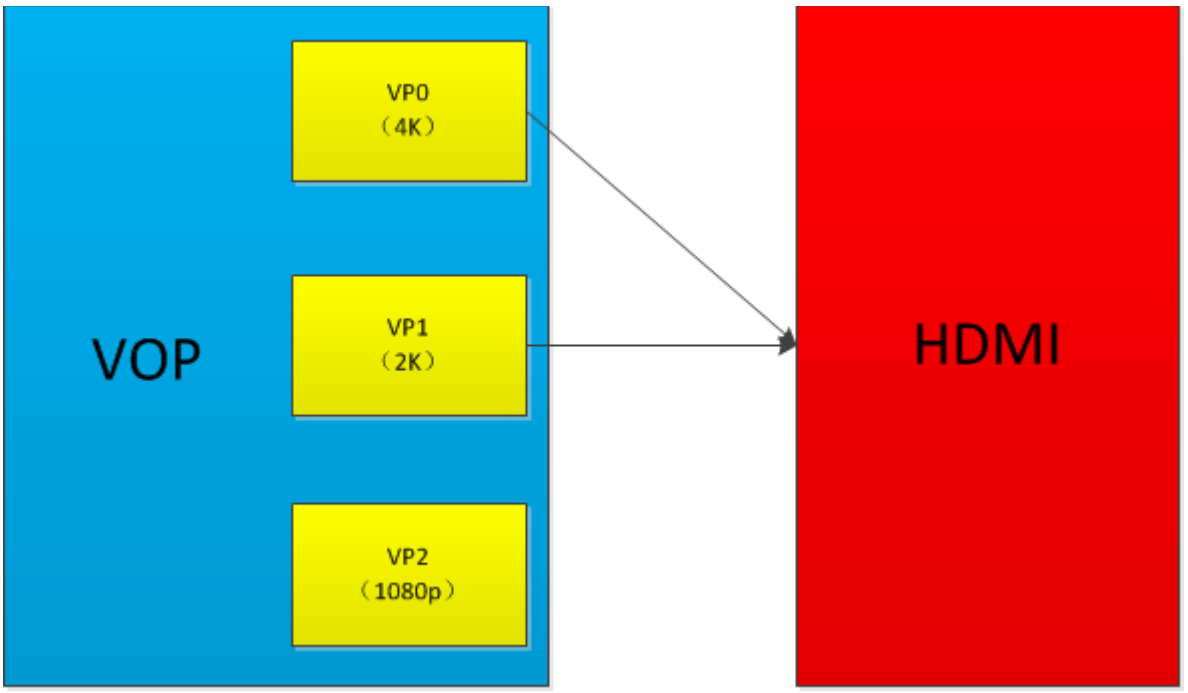
Conversely, if bound to VOPL, add:

```
&hdmi_in_vopb {
    status = "disabled";
};
```

If the platform has only one VOP, this step is not necessary.

For VOP2 and later versions, a platform no longer has multiple VOPs. Instead, there is only one VOP with multiple VPs (Video Ports) outputting.

The RK356X VOP and HDMI path is shown in the figure:

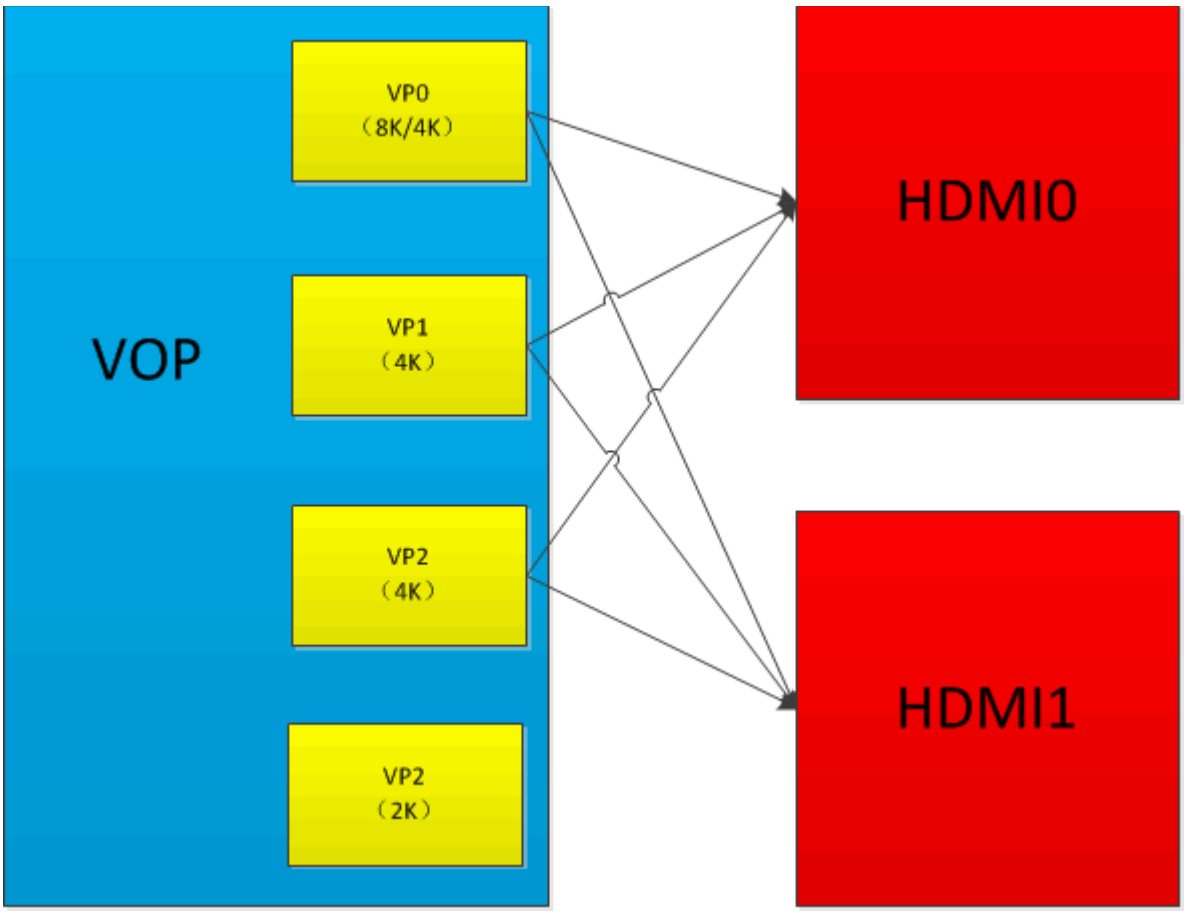


HDMI can be bound to VP0 or VP1, and it is recommended to bind to VP0 to support 4K output:

```
&hdmi_in_vp0 {
    status = "okay";
};

&hdmi_in_vp1 {
    status = "disabled";
};
```

The RK3588 VOP and HDMI path is shown in the figure:



RK3588 has two HDMITXs, which are completely identical in performance and can each be bound to VP0, VP1, or VP2.

If the highest resolution required is 4K, it is recommended to bind HDMI0/1 to VP0/1 respectively:

```
&hdmi0_in_vp0 {
    status = "okay";
};

&hdmi0_in_vp1 {
    status = "disabled";
};

&hdmi0_in_vp2 {
    status = "disabled";
};

&hdmi1_in_vp1 {
    status = "okay";
};

&hdmi1_in_vp0 {
    status = "disabled";
};

&hdmi1_in_vp2 {
    status = "disabled";
};
```

If the RK3588 platform needs to output an 8K resolution, it must occupy two ports, VP0 and VP1, for stitching. In DTS, the HDMI that outputs 8K must be bound to VP0, taking HDMI0 as an example:

```
&hdmi0_in_vp0 {
    status = "okay";
};

&hdmi0_in_vp1 {
    status = "disabled";
};

&hdmi0_in_vp2 {
    status = "disabled";
};
```

At the same time, it is also necessary to set the VOP ACLK to 800 MHz, see 2.1.4 for details.

RK3588 HDMI shares the COMBPHY with eDP, taking HDMI0 and eDP0 as examples, DTS hdptxphy_hdmi0 is the HDMI PHY node, and hdptxphy0 is the eDP PHY node:

```
    hdptxphy0: phy@fed60000 {
        compatible = "rockchip,rk3588-hdptx-phy";
        reg = <0x0 0xfed60000 0x0 0x2000>;
        clocks = <&cru CLK_USB2PHY_HDPTXRXPHY_REF>, <&cru PCLK_HDPTX0>;
        clock-names = "ref", "apb";
        resets = <&cru SRST_P_HDPTX0>, <&cru SRST_HDPTX0_INIT>,
            <&cru SRST_HDPTX0_CMN>, <&cru SRST_HDPTX0_LANE>;
        reset-names = "apb", "init", "cmn", "lane";
        rockchip,grf = <&hdptxphy0_grf>;
        #phy-cells = <0>;
        status = "disabled";
    };

    hdptxphy_hdmi0: hdmiphy@fed60000 {
        compatible = "rockchip,rk3588-hdptx-phy-hdmi";
        reg = <0x0 0xfed60000 0x0 0x2000>;
        clocks = <&cru CLK_USB2PHY_HDPTXRXPHY_REF>, <&cru PCLK_HDPTX0>;
        clock-names = "ref", "apb";
        clock-output-names = "clk_hdmiphy_pixel0";
        #clock-cells = <0>;
        resets = <&cru SRST_HDPTX0>, <&cru SRST_P_HDPTX0>,
            <&cru SRST_HDPTX0_INIT>, <&cru SRST_HDPTX0_CMN>,
            <&cru SRST_HDPTX0_LANE>, <&cru SRST_HDPTX0_ROPLL>,
            <&cru SRST_HDPTX0_LCPLL>;
        reset-names = "phy", "apb", "init", "cmn", "lane", "ropll",
                "lcpll";
        rockchip,grf = <&hdptxphy0_grf>;
        #phy-cells = <0>;
        status = "disabled";
    };
```

Therefore, when using HDMI, the corresponding eDP and eDP PHY must be disabled. Taking HDMI0 as an example:

```
&hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi0 {
    status = "okay";
};

&edp0 {
    status = "disabled";
};

&hdptxphy0 {
    status = "disabled";
};
```

2.1.3 Enabling Boot Logo

If the U-Boot logo is not enabled, the kernel stage will also fail to display the boot logo, and the image displayed by the application can only be seen after the system starts. Enabling the U-Boot logo support in the `dts` file by setting `route_hdmi` to "okay":

```
&route_hdmi {
    status = "okay";
};
```

Additionally, on dual VOP platforms, it is important to ensure that the VOP specified in the `connect` field of the code below matches the VOP bound to HDMI (see section 2.1.2 for details), otherwise issues such as screen artifacts may arise.

```
route_hdmi: route-hdmi {
    status = "disabled";
    logo,uboot = "logo.bmp";
    logo,kernel = "logo_kernel.bmp";
    logo,mode = "center";
    charge_logo,mode = "center";
    connect = <&vopb_out_hdmi>;
};
```

2.1.4 Binding VOP dclk to PLL

HDMI-bound VOP/VP dclk requires specifying the corresponding PLL as the clock source. In the RK platform, RK322X/RK3328/RK3528 HDMI is fixedly bound to the VOP/VP and uses the HDMI PHY PLL as the dclk clock source, which does not require configuration.

2.1.4.1 RK3288 PLL Binding

The RK3288 VOPB/VOPL dclk can be connected to GPLL/CPLL. Since neither of these PLLs supports fractional frequency division, the RK3288 HDMI can only output standard resolutions derived via integer frequency division from a 594MHz PLL (such as 4K60/1080P60/720P60). For example, VOPB dclk is connected to GPLL, and VOPL dclk is connected to CPLL as follows:

```
&vopb {
	assigned-clocks = <&cru DCLK_VOP0>;
	assigned-clock-parents = <&cru PLL_GPLL>;
};

&vopl {
	assigned-clocks = <&cru DCLK_VOP1>;
	assigned-clock-parents = <&cru PLL_CPLL>;
};
```

2.1.4.2 RK3399 Binding PLL

The HDMI of RK3399 needs to bind the VOP dclk to VPLL, and if it's dual display, another VOP dclk should be mounted to CPLL. This allows for the generation of arbitrary frequency dclk, enabling support for any resolution in dual display. For example, when HDMI is bound to VOPB, the configuration is as follows:

```
&vopb {
   assigned-clocks = <&cru DCLK_VOP0_DIV>;
   assigned-clock-parents = <&cru PLL_VPLL>;
};
&vopl {
   assigned-clocks = <&cru DCLK_VOP1_DIV>;
   assigned-clock-parents = <&cru PLL_CPLL>;
};
```

When HDMI is bound to VOPL, the configuration is:

```
&vopb {
   assigned-clocks = <&cru DCLK_VOP0_DIV>;
   assigned-clock-parents = <&cru PLL_CPLL>;
};
&vopl {
   assigned-clocks = <&cru DCLK_VOP1_DIV>;
   assigned-clock-parents = <&cru PLL_VPLL>;
};
```

2.1.4.3 RK356X Binding PLL

The VP dclk of RK356X HDMI must be mounted on HPLL, for example, VP0 dclk:

```
&vop {
	status = "okay";
	assigned-clocks = <&cru DCLK_VOP0>;
	assigned-clock-parents = <&pmucru PLL_HPLL>;
};
```

2.1.4.4 RK3588/RK3576 Binding PLL

For RK3588 to output resolutions above 4K60, the VOP ACLK should be set to 800MHz:

```
&vop {
	assigned-clocks = <&cru ACLK_VOP>;
	assigned-clock-rates = <800000000>;
	status = "okay";
};
```

RK3576 VOP ACLK does not require specific configuration.

RK3588 HDMI 0/1 can be bound to VP 0/1/2 as detailed in section 2.1.2. The dclk of these three VPs can all be connected to GPLL/HDMI0 PHY PLL/HDMI 1 PHY PLL/V0PLL.

Specific allocation strategies and related restrictions can be referred to in section 10.11 of the document **Rockchip_Developer_Guide_DRM_Display_Driver_EN.pdf**.

If RK3588/RK3576 needs to support non-standard resolutions, a PHY PLL must be specified as the VOP dclk clock source.

RK3588 DTS configuration is as follows:

```
&display_subsystem {
   clocks = <&hdptxphy_hdmi_clk0>, <&hdptxphy_hdmi_clk1>;
   clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};

&hdptxphy_hdmi_clk0 {
   status = "okay";
};

&hdptxphy_hdmi_clk1 {
   status = "okay";
};
```

RK3576 DTS configuration is as follows:

```
&display_subsystem {
    clocks = <&hdptxphy_hdmi>;
    clock-names = "hdmi0_phy_pll";
};
```

The effectiveness of DTS clock allocation can be confirmed using the command `cat /sys/kernel/debug/clk/clk_summary`.

Taking the dclk of VP0 bound to HDMI0 PHY PLL as an example, `dclk_vop0` is mounted under `clk_hdmiphy_pixel0` in the clock tree:

```
 xin24m                        26      28       0    24000000         0     0  50000
    clk_usbdpphy_mipidcpphy_ref      2       2       0    24000000         0     0  50000
    clk_usb2phy_hdptxrxphy_ref      11      11       0    24000000         0     0  50000
       clk_hdmiphy_pixel0            2       3       0   594000000         0     0  50000
          dclk_vop0                  2       4       0   594000000         0     0  50000
```

2.1.5 Enabling HDCP for RK3288/RK3399/RK3528/RK356X

2.1.5.1 Enable HDCP 1.4

```
&hdmi {
    hdcp1x-enable = <1>;
};
```

After enabling HDCP 1.4, it is necessary to program the corresponding key using a tool, which can be obtained in the RKTools directory of the SDK. Different Android tools may vary, so consult with FAE for details, and refer to the tool's readme for usage instructions. The corresponding Key must be applied for by the customer from Digital Content Protection LLC.

The HDCP function can be enabled or disabled through the following node:

```
echo 1 > /sys/class/misc/hdmi_hdcp1x/enable
```

A value of 1 enables the HDCP function, while 0 disables it.

After enabling the HDCP function, you can confirm whether HDCP is effective by the following methods:

- Check the HDCP working status through the following node:

  ```
  cat /sys/class/misc/hdmi_hdcp1x/status
  ```

  The different values correspond to the following HDCP statuses:

status node value	Description
hdcp disable	HDCP function is disabled.
hdcp_auth_start	HDCP begins authentication.
hdcp_auth_success	HDCP authentication is successful, encrypted video data transmission starts.
hdcp_auth_fail	HDCP authentication fails.

- Test with a TV that does not support HDCP 1.4 and one that does. If, after enabling HDCP, the TV that does not support HDCP 1.4 displays a pink screen while the one that supports HDCP 1.4 displays normally, it indicates that HDCP is functioning correctly.

2.1.5.2 Enabling HDCP 2.2

RK3288/RK3399/RK3528/RK356X support HDCP 2.2 functionality under the DRM framework. It is important to note that to use HDCP 2.2, HDCP 1.4 must be functioning properly. To enable this feature, follow these steps:

- Request the HDCP 2.2 Key packaging tool and patch package from FAE, and package the Key and apply the patch according to the readme.
- After recompiling and flashing, use the following node to enable/disable HDCP 2.2 functionality:

```
echo 1 > /sys/class/misc/hdcp2_node/enable
```

After enabling, you can determine if HDCP 2.2 is working properly by:

- Finding a TV that does not support HDCP 2.2 and one that does. If, after enabling HDCP, the TV that does not support HDCP 2.2 displays a white screen while the TV that supports HDCP 2.2 displays normally, this indicates that HDCP is functioning correctly.
- Obtain the HDCP 2.2 working status through the following node:

```
cat /sys/class/misc/hdcp2_node/status
```

status value	Description
hdcp2 auth success	Authentication successful.
no enable hdcp2	HDCP 2.2 is disabled.
hdcp2 no auth	HDMI is not connected or the device does not support HDCP 2.2.
no already auth success	Authentication failed.

- If authentication failure occurs, please upload the following Log files on redmine:

```
/cache/hdcp_tx0.log
```

  Or execute the following commands to capture the Log:

```
logcat -s HDMI_HDCP2
dmesg | grep hdcp
```

2.1.6 Enabling HDCP on RK3588/RK3576

The RK3588/RK3576 supports HDCP 1.4/2.3, and enabling either requires invoking the HDCP interface of the DRM framework. HDCP can be toggled and its status queried by configuring DRM PROPERTY. The relevant demo code, hdcptest.c, should be requested from the FAE. For debugging, modetest can be utilized for testing.

The Android system modetest code path is:

```
external/libdrm/tests/modetest/
```

For Linux systems, modetest compiled with buildroot can be used. After compiling the rootfs, the code path is:

```
buildroot/output/rockchip_rk3588/build/libdrm-2.4.115/tests/modetest/
```

To test HDCP with modetest, the following properties are required:

```
Content Protection:
        flags: enum
        enums: Undesired=0 Desired=1 Enabled=2
        value: 2

Undesired: Disable HDCP
Desired: Enable HDCP
Enabled: HDCP is enabled and authenticated successfully

hdcp_encrypted:
        flags: range
        values: 0 2
        value: 2

HDCP Authentication Levels:
  0: HDCP not authenticated.
  1: HDCP 1.4.
  2: HDCP 2.3.
```

The HDCP switch can be controlled using the command `modetest -w`, with examples provided below:

```
modetest -w 423:"Content Protection":1
```

Enable HDCP, prioritize 2.3; if the TV does not support 2.3 or 2.3 authentication fails, automatically switch to 1.4.

```
modetest -w 423:"Content Protection":0
```

Disable HDCP.

The number 423 represents the HDMI `connector id`, which can be queried using the command `modetest -c`:

```
rk3588_s:/ # modetest -c
trying to open device 'i915'...failed
trying to open device 'amdgpu'...failed
trying to open device 'radeon'...failed
trying to open device 'nouveau'...failed
trying to open device 'vmwgfx'...failed
trying to open device 'omapdrm'...failed
trying to open device 'exynos'...failed
trying to open device 'tilcdc'...failed
trying to open device 'msm'...failed
trying to open device 'sti'...failed
trying to open device 'tegra'...failed
trying to open device 'imx-drm'...failed
trying to open device 'rockchip'...done
Connectors:
id      encoder status          name            size (mm)       modes   encoders
423     422     connected       HDMI-A-1        0x0             6       422
  modes:
        name refresh (Hz) hdisp hss hse htot vdisp vss vse vtot)
  1920x1080 60 1920 2008 2052 2200 1080 1084 1089 1125 148500 flags: phsync, pvsync; type: preferred
  1920x1080 50 1920 2448 2492 2640 1080 1084 1089 1125 148500 flags: phsync, pvsync; type: driver
  1280x720 60 1280 1390 1430 1650 720 725 730 750 74250 flags: phsync, pvsync; type: driver
  1280x720 50 1280 1720 1760 1980 720 725 730 750 74250 flags: phsync, pvsync; type: driver
  720x576 50 720 732 796 864 576 581 586 625 27000 flags: nhsync, nvsync; type: driver
  720x480 60 720 736 798 858 480 489 495 525 27000 flags: nhsync, nvsync; type: driver
```

For detailed instructions on using HDCP, refer to the document **Rockchip_RK3588_Developer_Guide_HDCP_EN.pdf**.

2.1.7 DDC I2C Speed Configuration

When HDMI hot-plugging fails to read EDID, you can try to reduce the DDC I2C speed.

```
[  163.026743] dwhdmi-rockchip fdea0000.hdmi: i2c read time out!
[  163.130075] dwhdmi-rockchip fdea0000.hdmi: i2c read time out!
[  163.233407] dwhdmi-rockchip fdea0000.hdmi: i2c read time out!
[  163.336741] dwhdmi-rockchip fdea0000.hdmi: i2c read time out!
[  163.440074] dwhdmi-rockchip fdea0000.hdmi: i2c read time out!
[  163.440110] dwhdmi-rockchip fdea0000.hdmi: failed to get edid
```

Currently, the I2C speed is adjusted by the duration of high and low levels of the clk signal. The following is the actual measured I2C speed configuration at 50 kHz.

```
&hdmi {
    ddc-i2c-scl-high-time-ns = <9625>;
    ddc-i2c-scl-low-time-ns = <10000>;
};
```

To adjust the I2C speed, simply modify these two values in proportion. For example, to adjust the speed to 100 kHz:

```
&hdmi {
    ddc-i2c-scl-high-time-ns = <4812>;
    ddc-i2c-scl-low-time-ns = <5000>;
};
```

2.1.8 HDMI Signal Strength Configuration

Due to differences in hardware wiring, different boards may require different driver strength configurations. When encountering TV compatibility issues, you can try modifying the settings.

2.1.8.1 Signal Strength Configuration for RK322X/RK3328/RK3528/RK356X

2.1.8.1.1 RK322X Signal Configuration

The HDMI signal strength of RK322X can be configured through the `rockchip,phy-table` attribute in dts, with the following format definition:

```
    rockchip,phy-table =
        <165000000 0xaa 0x00 0x44 0x44 0x00 0x00 0x00
            0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00>,
        <340000000 0xaa 0x15 0x6a 0xaa 0x00 0x00 0x00
            0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00>,
        <594000000 0xaa 0x15 0x7a 0xaa 0x00 0x00 0x00
            0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00>;
```

Taking the 340000000 row as an example:

Parameter	Description
340000000	Represents the maximum TMDS clock frequency corresponding to this row of parameters. This row of configuration is applicable to resolutions with a TMDS clock lower than 340MHz
0xaa	Data lane0 slew rate: bit[1:0] Data lane1 slew rate: bit[3:2] Data lane2 slew rate: bit[5:4] Clock lane slew rate: bit[7:6] Slew rate: Adjusts the rise and fall time, the larger the value, the shorter the time.
0x15	Data lane0 pre-emphasis: bit[1:0] Data lane1 pre-emphasis: bit[3:2] Data lane2 pre-emphasis: bit[5:4] Pre-emphasis: The larger the value, the greater the pre-emphasis.
0x6a	Data lane2 swing: bit[3:0] Clock lane swing: bit[7:4] Swing: Amplitude, the larger the value, the greater the amplitude.
0xaa	Data lane0 swing: bit[3:0] Data lane1 swing: bit[7:4] Swing: Amplitude, the larger the value, the greater the amplitude.
Subsequent 0s	Invalid

2.1.8.1.2 RK3328 Signal Configuration

The HDMI signal strength of RK3328 can be configured through the `rockchip,phy-table` attribute in dts, with the format defined as follows:

```
rockchip,phy-table =
        <165000000 0x07 0x0a 0x0a 0x0a 0x00 0x00 0x08
                0x08 0x08 0x00 0xac 0xcc 0xcc 0xcc>,
        <340000000 0x0b 0x0d 0x0d 0x0d 0x07 0x15 0x08
                0x08 0x08 0x3f 0xac 0xcc 0xcd 0xdd>,
        <594000000 0x10 0x1a 0x1a 0x1a 0x07 0x15 0x08
                0x08 0x08 0x00 0xac 0xcc 0xcc 0xcc>;
```

Taking the 340000000 row as an example:

Parameter	Description
340000000	Represents the maximum TMDS clock frequency corresponding to the parameters in this row. That is, the configuration in this row is applicable to resolutions with a TMDS clock below 340MHz
0x0b	Clock lane swing Swing: Amplitude, the larger the value, the greater the amplitude.
0x0d	Data lane2 swing Swing: Amplitude, the larger the value, the greater the amplitude.
0x0d	Data lane1 swing Swing: Amplitude, the larger the value, the greater the amplitude.
0x0d	Data lane0 swing Swing: Amplitude, the larger the value, the greater the amplitude.
0x07	Data lane0 pre-emphasis mode: bit[0] Data lane1 pre-emphasis mode: bit[1] Data lane2 pre-emphasis mode: bit[2] Pre-emphasis mode: 0--full mode 1--half mode
0x15	Data lane0 pre-emphasis level: bit[1:0] Data lane1 pre-emphasis level: bit[3:2] Data lane2 pre-emphasis level: bit[5:4] Pre-emphasis level: Pre-emphasis strength, the larger the value, the greater the emphasis, and it is turned off when it is 0.
0x08	Data lane2 pre-emphasis swing: bit[4:0] Pre-emphasis swing: Pre-emphasis amplitude, the larger the value, the greater the amplitude.
0x08	Data lane1 pre-emphasis swing: bit[4:0] Pre-emphasis swing: Pre-emphasis amplitude, the larger the value, the greater the amplitude.
0x08	Data lane0 pre-emphasis swing: bit[4:0] Pre-emphasis swing: Pre-emphasis amplitude, the larger the value, the greater the amplitude.
0x3f	Data lane0 pre-emphasis driver path gate: bit[0], main driver path gate: bit[1] Data lane1 pre-emphasis driver path gate: bit[2], main driver path gate: bit[3] Data lane2 pre-emphasis driver path gate: bit[4], main driver path gate: bit[5] Clock lane pre-emphasis driver path gate: bit[6], main driver path gate: bit[7] Write 1 to enable the path.
0xac	Data lane2 pre-emphasis second delay gate: bit[0], pre-emphasis first delay gate: bit[1] Data lane2 main driver second delay gate: bit[2], main driver first delay gate: bit[3] Clock lane pre-emphasis second delay gate: bit[4], pre-emphasis first delay gate: bit[5] Clock lane main driver second delay gate: bit[6], main driver first delay gate: bit[7] Write 1 to enable delay.

Parameter	Description
0xcc	Data lane0 pre-emphasis second delay gate: bit[0], pre-emphasis first delay gate: bit[1] Data lane0 main driver second delay gate: bit[2], main driver first delay gate: bit[3] Data lane1 pre-emphasis second delay gate: bit[4], pre-emphasis first delay gate: bit[5] Data lane1 main driver second delay gate: bit[6], main driver first delay gate: bit[7] Write 1 to enable delay.
0xcd	Data lane2 delay time: bit[3:0] Clock lane delay time: bit[7:4] Delay time: Delay duration, the larger the value, the greater the delay.
0xdd	Data lane0 delay time: bit[3:0] Data lane1 delay time: bit[7:4] Delay time: Delay duration, the larger the value, the greater the delay.

2.1.8.1.3 RK3528 Signal Configuration

The HDMI signal strength of RK3528 can be configured through the `rockchip,phy-table` attribute in dts, with the format defined as follows:

```
rockchip,phy-table =
        <165000000 0x03 0x04 0x0c 0x12 0x00 0x00 0x00
                0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00>,
        <340000000 0x03 0x04 0x0c 0x12 0x00 0x00 0x00
                0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00>,
        <594000000 0x02 0x08 0x0d 0x18 0x00 0x00 0x00
                0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00>;
```

Taking the 340000000 row as an example:

Parameter	Description
340000000	Represents the maximum TMDS clock frequency corresponding to the parameters in this row. That is, the configuration in this row is applicable to resolutions with a TMDS clock below 340MHz
0x03	Clock lane current bias control: 0x00:320uA 0x0f:920uA Step of 40uA
0x04	Data lane current bias control: 0x00:320uA 0x0f:920uA Step of 40uA
0x0c	Clock lane swing: bit[4:0] Swing: amplitude, the larger the value, the larger the amplitude.
0x12	Data lane swing: bit[4:0] ESD event detection threshold: bit[7:5] Swing: amplitude, the larger the value, the larger the amplitude. ESD event detection threshold: Triggers the ESD event threshold, the larger the value, the higher the threshold.
0x00	Pre-cursor pre-emphasis: bit[2:0] Post-cursor pre-emphasis: bit[7:4] Pre-emphasis: Pre-emphasis, the larger the value, the stronger the pre-emphasis, 0 value turns off pre-emphasis.
All subsequent 0 values	Invalid

2.1.8.2 RK3288/RK3368/RK3399/RK356X Signal Configuration

The HDMI signal strength of RK3288/RK3368/RK3399/RK356X can be configured through the `rockchip,phy-table` attribute in dts, with the format defined as:

<PIXELCLOCK PHY_CKSYMTXCTRL PHY_TXTERM PHY_VLEVCTRL>

PIXELCLOCK indicates the maximum pixelclock frequency corresponding to the parameters in this row.

PHY_CKSYMTXCTRL register (0x09) value is used to adjust the pre-emphasis and rise slope of the HDMI signal. Increasing pre-emphasis or slope boost can enhance the rise/fall slope of the DATA signal, but will reduce the signal's rise/fall time:

Bit[0]: CLOCK signal enable.
Bit[3:1]: DATA signal pre-emphasis.
Bit[5:4]: DATA signal slope boost.

PHY_TXTERM register (0x19) value is used to adjust the termination resistance value.

Bit[0:2]: The larger the value, the greater the termination resistance value.

PHY_VLEVCTRL register (0x0e) value is used to adjust the HDMI signal amplitude, specifically defined as follows: Bit[0:4] : tmds_clk +/- signal amplitude, the lower the value, the greater the signal amplitude; Bit[5:9]: tmds_data +/- signal amplitude, the lower the value, the greater the signal amplitude.

Example as follows:

```
&hdmi {
rockchip,phy-table =
   <74250000 0x8009 0x0004 0x0272>,
   <165000000 0x802b 0x0004 0x0209>,
   <297000000 0x8039 0x0005 0x028d>,
   <594000000 0x8039 0x0000 0x019d>,
   <000000000 0x0000 0x0000 0x0000>;
};
```

Where <74250000 0x8009 0x0004 0x0272> indicates that the pixel clock is 74.25MHz (720p resolution) and below, with PHY_CKSYMTXCTRL register value as 0x8009; PHY_TXTERM value as 0x0004; PHY_VLEVCTRL value as 0x0272. After modification, you can also use the `cat /sys/kernel/debug/dw-hdmi/phy` command to check the corresponding register values to confirm whether the changes have been successful.

2.1.9 Added Special Resolutions

2.1.9.1 Adding Special Resolution Timings

The DRM framework currently supports the vast majority of resolution timings, but there may still be some special resolutions not supported in scenarios where HDMI screens are rotated. New items need to be added at the end of `drm_dmt_modes` in `kernel/drivers/gpu/drm/drm_edid.c`:

```
   /* 0x58 - 4096x2160@59.94Hz RB */
   { DRM_MODE("4096x2160", DRM_MODE_TYPE_DRIVER, 556188, 4096, 4104,
        4136, 4176, 0, 2160, 2208, 2216, 2222, 0,
        DRM_MODE_FLAG_PHSYNC | DRM_MODE_FLAG_NVSYNC) },
```

Parameter	Description
"4096x2160"	mode name, representing the hdisplay x vdisplay resolution
DRM_MODE_TYPE_DRIVER	mode type, configured as DRM_MODE_TYPE_DRIVER
556188	pixel clock
4096	horizontal active pixels
4104	horizontal sync start pixel
4136	horizontal sync end pixel
4176	total horizontal pixels
0	hskew, usually 0
2160	frame active lines
2208	frame sync start line
2216	frame sync end line
2222	total frame lines
0	vscan, usually 0
DRM_MODE_FLAG_PHSYNC | DRM_MODE_FLAG_NVSYNC	hsync and vsync polarity, flag definitions are as follows: DRM_MODE_FLAG_PHSYNC (1<<0) DRM_MODE_FLAG_NHSYNC (1<<1) DRM_MODE_FLAG_PVSYNC (1<<2) DRM_MODE_FLAG_NVSYNC (1<<3) DRM_MODE_FLAG_INTERLACE (1<<4)

The descriptions of each parameter are as shown in the table above, and the specific timing meanings can be referenced in section 2.1.10.1.

2.1.9.2 New PLL Configuration for RK322X/RK3328/RK3528

The RK322X/RK3328/RK3528 chips require additional HDMI-PHY-PLL configurations for special resolutions. The PLL parameter calculation is divided into two parts: PRE-PLL and POST-PLL.

2.1.9.2.1 PRE-PLL Calculation Process

The PRE-PLL calculation process is as follows, where F_{REF} is 24MHz:

The VCO frequency of PRE-PLL ($F_{pre\text{-}VCO}$) is controlled by `pre-pll-pre-divider[5:0]` and `pre-pll-feedback-divider[11:0]`. For the RK3328 chip, it can also be controlled by `pre-pll-fractional-feedback-divider[23:0]`, supporting floating-point calculation.

$$F_{pre-VCO} = \frac{F_{REF}}{pre\text{–}pll\text{–}pre\text{–}divider[5:0]} \times (pre\text{–}pll\text{–}feedback\text{–}divider[11:0] + \frac{pre\text{–}pll\text{–}fractional\text{–}feedback\text{–}divider[23:0]}{2^{24}})$$

The TMDS CLOCK channel frequency F_{TX3} is controlled by `tmds-dividera[1:0]` and `tmds-dividerb[1:0]`.

$$F_{TX3} = \frac{F_{pre-VCO}}{4 \times tmds\text{–}dividera[1:0] \times tmds\text{–}dividerb[1:0]}$$

$$tmds\text{–}dividera[1:0] = 1, 2, 3, 5$$
$$tmds\text{–}dividerb[1:0] = 1, 2, 4, 8$$

The `pin_hdmi20_tmdsclk` frequency $F_{pin_hdmi20_tmdsclk}$ is controlled by `tmds-dividera[1:0]` and `tmds-dividerc[1:0]`.

$$F_{pin_hdmi20_tmdsclk} = \frac{F_{pre-VCO}}{4 \times tmds\text{–}dividera[1:0] \times tmds\text{–}dividerc[1:0]}$$

$$tmds\text{–}dividera[1:0] = 1, 2, 3, 5$$
$$tmds\text{–}dividerc[1:0] = 1, 2, 4, 8$$

The `pin_hdmi20_prepclk` frequency $F_{pin_hdmi20_prepclk}$ is calculated as follows:

$$F_{pin_hdmi20_prepclk} = (pclk\text{–}dividera[4:0] == 1)\ ?$$

$$\frac{F_{pre-VCO}}{pclk\text{–}dividerb[1:0] \times pclk\text{–}dividerc[1:0]}\ :$$

$$\frac{F_{pre-VCO}}{pclk\text{–}dividera[4:0] \times pclk\text{–}dividerc[1:0]}$$

$$pclk\text{–}dividera[4:0] = 1 \sim 31$$
$$pclk\text{–}dividerb[1:0] = 2, 3, 4, 5$$
$$pclk\text{–}dividerc[1:0] = 1, 2, 4, 8$$

When the `pin_hdmi20_pclk` frequency ($F_{pin_hdmi20_pclk}$) is 1/5 of $F_{pre\text{-}VCO}$, VCO/5 can be enabled to directly obtain `pin_hdmi20_pclk`. Other frequency calculations are as follows:

$$F_{pin_hdmi20_pclk} = (pclk\text{–}dividera[4:0] == 1)\ ?$$

$$\frac{F_{pre-VCO}}{pclk\text{–}dividerb[1:0] \times pclk\text{–}dividerc[1:0]) \times 2}\ :$$

$$\frac{F_{pre-VCO}}{pclk\text{–}dividera[4:0] \times pclk\text{–}dividerc[1:0]) \times 2}$$

$$pclk\text{–}dividera[4:0] = 1 \sim 31$$
$$pclk\text{–}dividerb[1:0] = 2, 3, 4, 5$$
$$pclk\text{–}dividerd[4:0] = 1 \sim 31$$

Notes:

1. pin_hd20_tmdsclk is the TMDS CLOCK output by HDMI, and pin_hd20_pclk is the PIXEL CLOCK output by HDMI.
2. The pin_hd20_pclk frequency is obtained by dividing pin_hd20_prepclk by a factor of 1~10. In no-repeating mode, the two frequencies are equal.
3. The frequencies of pin_hd20_prepclk and pin_hd20_tmdsclk are equal in 8-bit color depth. In 10-bit color depth, pin_hd20_tmdsclk is 1.25 times the pin_hd20_prepclk frequency. Currently, the RK platform only supports 8-bit and 10-bit color depths.

As stated in point 2 above, in actual use, for non-4K-YUV420 scenarios, pin_hd20_pclk is always equal to pin_hd20_prepclk. In 4K-YUV420 scenarios, pin_hd20_pclk is twice pin_hd20_prepclk.

2.1.9.2.2 POST-PLL Calculation Process

The POST-PLL calculation process is as follows:

POST-PLL is used to generate the differential serial clock, and its frequency $F_{diff\text{-}sclk}$ is always five times pin_hdmi20_tmdsclk.

$$F_{post-VCO} = \frac{F_{pin_hdmi20_tmdsclk}}{post-pll-pre-divider[4:0]} \times post-pll-feedback-divider[8:0]$$

When post-pll-post-divider is disabled (0xaa[3:2] = 0), $F_{diff\text{-}sclk}$ is equal to $F_{post\text{-}VCO}$. Otherwise, the calculation is:

$$F_{diff-sclk} = \frac{F_{post-VCO}}{post-pll-post-divider[1:0]}$$

$$post-pll-post-divider = 2, 4, 8(2'b0, 2'b01, 2'b11)$$

The POST-PLL calculation is relatively simple, and the post_pll_cfg_table in the DRM framework driver already covers all scenarios, so no additional configuration is needed. For details on how to select the configuration from post_pll_cfg_table based on TMDS CLOCK and chip version, see the POST-PLL configuration selection explanation below.

Note that for both PRE-PLL and POST-PLL, the VCO frequency must be between 1.4~3.0GHz (which can be relaxed to 3.2GHz in actual use).

2.1.9.2.3 Using the Calculation Tool

In practice, the calculation tool cal_innophy can be used for calculations. This tool can be obtained through the FAE window.

Usage:

```
cal_innophy 148500000 185625000 1
```

The three parameters have the following meanings:

Parameter	Description
148500000	PIXEL CLOCK
185625000	TMDS CLOCK
1	Whether to use floating-point calculation. RK322X series chips do not support floating-point and can only be 0

For the third parameter, it is recommended to prioritize 0 (no floating-point). Only set it to 1 when the desired frequency cannot be achieved without floating-point.

The ratio of TMDS CLOCK to PIXEL CLOCK varies under different color depths.

The calculation result is:

```
148500000, 185625000, 4, 495, 0, 2, 2, 1, 3, 2, 2, 0, 0x816817
```

The meaning of each parameter is:

Parameter	Description
148500000	pixel clock
185625000	tmds clock
4	pre-pll-pre-divider
495	pre-pll-feedback-divider
0	tmds-dividera
2	tmds-dividerb
2	tmds-dividerc
1	pclk-dividera
3	pclk-dividerb
2	pclk-dividerc
2	pclk-dividerd
0	vco_div_5_en
0x816817	pre-pll-fractional-feedback-divider

When the DRM framework needs to add new PHY configurations, they must be added to the PRE-PLL configuration TABLE: `pre_pll_cfg_table`, while the POST-PLL configuration TABLE: `post_pll_cfg_table` already covers all resolutions supported by the PHY, and no additional configurations are needed. The path is:

```
kernel/drivers/phy/rockchip/phy-rockchip-inno-hdmi-phy.c
```

```
static const struct pre_pll_config pre_pll_cfg_table[] = {
    { 27000000,  27000000, 1,  90, 3, 2, 2, 10, 3, 3,  4, 0, 0},
    { 27000000,  33750000, 1,  90, 1, 3, 3, 10, 3, 3,  4, 0, 0},
    { 40000000,  40000000, 1,  80, 2, 2, 2, 12, 2, 2,  2, 0, 0},
    { 40000000,  50000000, 1, 100, 2, 2, 2,  1, 0, 0, 15, 0, 0},
    { 59341000,  59341000, 1,  98, 3, 1, 2,  1, 3, 3,  4, 0, 0xE6AE6B},
    { 59400000,  59400000, 1,  99, 3, 1, 1,  1, 3, 3,  4, 0, 0},
    { 59341000,  74176250, 1,  98, 0, 3, 3,  1, 3, 3,  4, 0, 0xE6AE6B},
    { 59400000,  74250000, 1,  99, 1, 2, 2,  1, 3, 3,  4, 0, 0},
    { 65000000,  65000000, 1, 130, 2, 2, 2,  1, 0, 0, 12, 0, 0},
    { 65000000,  81250000, 3, 325, 0, 3, 3,  1, 0, 0, 10, 0, 0},
    { 71000000,  71000000, 3, 284, 0, 3, 3,  1, 0, 0,  8, 0, 0},
    { 71000000,  88750000, 3, 355, 0, 3, 3,  1, 0, 0, 10, 0, 0},
    { 74176000,  74176000, 1,  98, 1, 2, 2,  1, 2, 3,  4, 0, 0xE6AE6B},
    { 74250000,  74250000, 1,  99, 1, 2, 2,  1, 2, 3,  4, 0, 0},
    { 74176000,  92720000, 4, 494, 1, 2, 2,  1, 3, 3,  4, 0, 0x816817},
    { 74250000,  92812500, 4, 495, 1, 2, 2,  1, 3, 3,  4, 0, 0},
    { 83500000,  83500000, 2, 167, 2, 1, 1,  1, 0, 0,  6, 0, 0},
    { 83500000, 104375000, 1, 104, 2, 1, 1,  1, 1, 0,  5, 0, 0x600000},
    { 85750000,  85750000, 3, 343, 0, 3, 3,  1, 0, 0,  8, 0, 0},
    { 88750000,  88750000, 3, 355, 0, 3, 3,  1, 0, 0,  8, 0, 0},
    { 88750000, 110937500, 1, 110, 2, 1, 1,  1, 1, 0,  5, 0, 0xF00000},
    {108000000, 108000000, 1,  90, 3, 0, 0,  1, 0, 0,  5, 0, 0},
    {108000000, 135000000, 1,  90, 0, 2, 2,  1, 0, 0,  5, 0, 0},
    {119000000, 119000000, 1, 119, 2, 1, 1,  1, 0, 0,  6, 0, 0},
    {119000000, 148750000, 1,  99, 0, 2, 2,  1, 0, 0,  5, 0, 0x2AAAAA},
    {148352000, 148352000, 1,  98, 1, 1, 1,  1, 2, 2,  2, 0, 0xE6AE6B},
    {148500000, 148500000, 1,  99, 1, 1, 1,  1, 2, 2,  2, 0, 0},
    {148352000, 185440000, 4, 494, 0, 2, 2,  1, 3, 2,  2, 0, 0x816817},
    {148500000, 185625000, 4, 495, 0, 2, 2,  1, 3, 2,  2, 0, 0},
    {162000000, 162000000, 1, 108, 0, 2, 2,  1, 0, 0,  4, 0, 0},
    {162000000, 202500000, 1, 135, 0, 2, 2,  1, 0, 0,  5, 0, 0},
    {296703000, 296703000, 1,  98, 0, 1, 1,  1, 0, 2,  2, 0, 0xE6AE6B},
    {297000000, 297000000, 1,  99, 0, 1, 1,  1, 0, 2,  2, 0, 0},
    {296703000, 370878750, 4, 494, 1, 2, 0,  1, 3, 1,  1, 0, 0x816817},
    {297000000, 371250000, 4, 495, 1, 2, 0,  1, 3, 1,  1, 0, 0},
    {593407000, 296703500, 1,  98, 0, 1, 1,  1, 0, 2,  1, 0, 0xE6AE6B},
    {594000000, 297000000, 1,  99, 0, 1, 1,  1, 0, 2,  1, 0, 0},
    {593407000, 370879375, 4, 494, 1, 2, 0,  1, 3, 1,  1, 1, 0x816817},
    {594000000, 371250000, 4, 495, 1, 2, 0,  1, 3, 1,  1, 1, 0},
    {593407000, 593407000, 1,  98, 0, 2, 0,  1, 0, 1,  1, 0, 0xE6AE6B},
    {594000000, 594000000, 1,  99, 0, 2, 0,  1, 0, 1,  1, 0, 0},
    {    ~0UL,     0, 0,   0, 0, 0, 0,  0, 0, 0,  0, 0, 0}
};

static const struct post_pll_config post_pll_cfg_table[] = {
    {33750000,  1, 40, 8, 1},
    {33750000,  1, 80, 8, 2},
    {33750000,  1, 10, 2, 4},
    {74250000,  1, 40, 8, 1},
    {74250000, 18, 80, 8, 2},
    {74250000,  1, 20, 4, 8},
    {148500000, 2, 40, 4, 3},
```

```
    {148500000, 1, 10, 2, 8},
    {297000000, 4, 40, 2, 3},
    {297000000, 2, 20, 2, 8},
    {594000000, 8, 40, 1, 3},
    {594000000, 4, 20, 1, 8},
    {    ~0UL, 0,  0, 0, 0}
};
```

The definitions of `struct pre_pll_config` and `struct post_pll_config` are as follows:

```
struct pre_pll_config {
    unsigned long pixclock;
    unsigned long tmdsclock;
    u8 prediv;
    u16 fbdiv;
    u8 tmds_div_a;
    u8 tmds_div_b;
    u8 tmds_div_c;
    u8 pclk_div_a;
    u8 pclk_div_b;
    u8 pclk_div_c;
    u8 pclk_div_d;
    u8 vco_div_5_en;
    u32 fracdiv;
};

struct post_pll_config {
    unsigned long tmdsclock;
    u8 prediv;
    u16 fbdiv;
    u8 postdiv;
    u8 version;
};
```

Explanation of `pre_pll_config` parameters is given in the table below:

Parameter	Description
pixclock	HDMI output resolution pixel clock
tmdsclock	HDMI output resolution tmds clock
prediv	pre-pll-pre-divider
fbdiv	pre-pll-feedback-divider
tmds_div_a	tmds-dividera
tmds_div_b	tmds-dividerb
tmds_div_c	tmds-dividerc
pclk_div_a	pclk-dividera
pclk_div_b	pclk-dividerb
pclk_div_c	pclk-dividerc
pclk_div_d	pclk-dividerd
vco_div_5_en	Whether pin_hd20_pclk is directly obtained from VCO / 5, used in specific clock scenarios
fracdiv	pre-pll-fractional-feedback-divider

Explanation of `post_pll_config` parameters is given in the table below:

Parameter	Description
tmdsclock	HDMI output resolution tmds clock
prediv	post-pll-pre-divider
fbdiv	post-pll-feedback-divider
postdiv	post-pll-post-divider
version	Chip version, POST-PLL configuration needs to be determined based on clock and chip version, the value meanings are: 1--RK322X and RK322XH early samples, tmds clock configurations for 74.25MHz and below 2--RK322XH mass production chips, tmds clock configurations for 74.25MHz and below 3--RK322X and RK322XH chips, tmds clock configurations above 74.25MHz, both configurations are the same 4--Some RK322X chips have unstable POST VCO at 1080MHz but stable at 270MHz, need to be distinguished 8--RK3528 specific configuration

Taking the TMDS CLOCK of 74.25MHz and the mass production chip of RK3328 as an example, the method to select the POST-PLL configuration is as follows:

1. First, find the corresponding range in `post_pll_cfg_table` based on TMDS CLOCK. For example, when TMDS CLOCK is 74.25MHz, 33.75MHz < TMDS CLOCK <= 74.25MHz, find the corresponding two items:

```
{74250000,  1, 40, 8, 1},
{74250000, 18, 80, 8, 2},
```

2. Further select based on the chip version. At this time, it is the mass production chip of RK3328, TMDS CLOCK <= 74.25MHz, so the value of version should choose 2, so the final selection is:

```
{74250000, 18, 80, 8, 2},
```

3. The final configuration values are: prediv = 18, fbdiv = 80, postdiv = 8. Since it is the mass production chip of RK3328 and TMDS CLOCK <= 74.25MHz, it needs to be added to `RK322XH_V1_PLL_TABLE`.

2.1.9.3 New PLL Configuration for RK3288/RK3368/RK3399/RK356X

The HDMI-PHY-PLL configurations for RK3288/RK3368/RK3399/RK356X are stored in `rockchip_mpll_cfg` and `rockchip_mpll_cfg_420`:

```
static const struct dw_hdmi_mpll_config rockchip_mpll_cfg[] = {
  {
    30666000, {
      { 0x00b3, 0x0000 },
      { 0x2153, 0x0000 },
      { 0x40f3, 0x0000 },
    },
  }, {
    36800000, {
      { 0x00b3, 0x0000 },
      { 0x2153, 0x0000 },
      { 0x40a2, 0x0001 },
    },
  }, {
    46000000, {
      { 0x00b3, 0x0000 },
      { 0x2142, 0x0001 },
      { 0x40a2, 0x0001 },
    },
  }, {
```

Path:

```
kernel/drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c
```

Where `rockchip_mpll_cfg` is the configuration for RGB/YUV444/YUV422, and `rockchip_mpll_cfg_420` is the configuration for YUV420.

The structure `dw_hdmi_mpll_config` is defined as follows:

```
struct dw_hdmi_mpll_config {
	unsigned long mpixelclock;
	struct {
		u16 cpce;
		u16 gmp;
	} res[DW_HDMI_RES_MAX];
};
```

Parameter descriptions are as follows:

Parameter	Description
mpixelclock	Pixel clock
cpce	OPMODE_PLLCFG register value
gmp	PLLGMPCTRL register value

Taking the first configuration item in `rockchip_mpll_cfg` as an example:

```
static const struct dw_hdmi_mpll_config rockchip_mpll_cfg[] = {
  {
    30666000, {
      { 0x00b3, 0x0000 },
      { 0x2153, 0x0000 },
      { 0x40f3, 0x0000 },
    },
  }, {
```

Firstly, the HDMI driver will determine whether the color format is YUV420. If so, it will select `rockchip_mpll_cfg_420`; otherwise, it will select `rockchip_mpll_cfg`. 30666000 indicates that the pixel clock of 30666000 and resolutions below apply to this configuration item. `{ 0x00b3, 0x0000 }`, `{ 0x2153, 0x0000 }`, `{ 0x40f3, 0x0000 }` correspond to configurations used for color depths of 8 BIT, 10 BIT, and 12 BIT (currently, the Rockchip solution only supports 8/10 bit modes).

Since the parameter values need to be obtained from the PHY's DATASHEET, if you need to add a new HDMI-PHY-PLL configuration, you can request the required pixel clock from the FAE. Then, according to the aforementioned rules, add the new configuration to `rockchip_mpll_cfg` or `rockchip_mpll_cfg_420`.

2.1.9.4 New PLL Configuration for RK3588/RK3576

The RK3588/RK3576 driver supports automatic calculation of PLL frequencies, eliminating the need for manual configuration when adding new resolutions.

Refer to section 10.11 of the document **Rockchip_Developer_Guide_DRM_Display_Driver_EN.pdf** for PLL allocation strategy details.

2.1.10 Open Audio

RK3368 and RK3288 default HDMI sound card and Codec share, configuration needs to be confirmed as follows:

```
&hdmi_analog_sound {
	status = "okay";
};
```

RK3399 currently shares the HDMI sound card with DP, and the configuration is as follows:

```
&hdmi_dp_sound {
	status = "okay";
};
```

2.2 Android Display Framework Configuration

Rockchip has added several system properties to the Android display framework to assist customers in configuring displays according to their needs.

2.2.1 Primary and Secondary Display Interface Configuration

Attribute	Function Description
sys.hwc.device.primary vendor.hwc.device.primary (After Android 9.0)	Set the display interface as the primary display
sys.hwc.device.extend vendor.hwc.device.extend (After Android 9.0)	Set the display interface as the secondary display

The configuration of these two attributes can be added to the system.prop file under the product configuration directory, such as:

```
device/rockchip/rk3368/rk3368_box/system.prop
```

By default (i.e., when the above attributes are not configured), devices that do not support hot-plugging (such as CVBS/MIPI/LVDS, etc.) will act as the primary display, while devices that support hot-plugging (such as HDMI/DP, etc.) will act as the secondary display.
Usually, only one display interface is configured for primary and secondary displays. For example, the RK3399 BOX SDK uses HDMI as the primary display and DP as the secondary display by default.

```
sys.hwc.device.primary=HDMI-A
sys.hwc.device.extend=DP
```

After 9.0, the attributes change to:

```
vendor.hwc.device.primary=HDMI-A
vendor.hwc.device.extend=DP
```

When multiple display interfaces are configured for primary/secondary displays, devices that support hot-plugging are prioritized. For example, the RK3368 BOX SDK uses the following default configuration:

```
sys.hwc.device.primary=HDMI-A,TV
```

After 9.0, the attribute changes to:

```
vendor.hwc.device.primary=HDMI-A,TV
```

When HDMI is plugged in, the primary display uses HDMI for display, and when HDMI is unplugged, the primary display uses CVBS for display.
Note: Since the framebuffer resolution of the primary display cannot be dynamically changed, it is best to set a framebuffer resolution for the primary display when two or more devices are used as the primary display.
For interface names, you can refer to the definitions in `hardware/rockchip/hwcomposer/drmresources.cpp`:

```
struct type_name connector_type_names[] = {
    { DRM_MODE_CONNECTOR_Unknown, "unknown" },//Unknown interface
    { DRM_MODE_CONNECTOR_VGA, "VGA" },  //VGA
    { DRM_MODE_CONNECTOR_DVII, "DVI-I" },//DVI, not supported temporarily
    { DRM_MODE_CONNECTOR_DVID, "DVI-D" },//DVI, not supported temporarily
    { DRM_MODE_CONNECTOR_DVIA, "DVI-A" },//DVI, not supported temporarily
    { DRM_MODE_CONNECTOR_Composite, "composite" },//Not supported
    { DRM_MODE_CONNECTOR_SVIDEO, "s-video" },//S-video
    { DRM_MODE_CONNECTOR_LVDS, "LVDS" },//LVDS
    { DRM_MODE_CONNECTOR_Component, "component" },//Component signal YPbPr
    { DRM_MODE_CONNECTOR_9PinDIN, "9-pin DIN" },//Not supported
    { DRM_MODE_CONNECTOR_DisplayPort, "DP" },//DP
    { DRM_MODE_CONNECTOR_HDMIA, "HDMI-A" },//HDMI Type A port
    { DRM_MODE_CONNECTOR_HDMIB, "HDMI-B" },//HDMI Type B port, not supported
    { DRM_MODE_CONNECTOR_TV, "TV" },// CVBS
    { DRM_MODE_CONNECTOR_eDP, "eDP" },//EDP
    { DRM_MODE_CONNECTOR_VIRTUAL, "Virtual" },//Not supported
    { DRM_MODE_CONNECTOR_DSI, "DSI" },//MIPI
};
```

2.2.2 Main and Auxiliary Display Interface Query

The names of the main and auxiliary display output interfaces can be queried through the following two read-only properties.

Property	Function Description
sys.hwc.device.main vendor.hwc.device.main (Used after Android 9.0)	Query the current main display output interface
sys.hwc.device.aux vendor.hwc.device.aux (Used after Android 9.0)	Query the current auxiliary display output interface

2.2.3 Framebuffer Resolution Configuration

The Framebuffer resolution is the resolution used for UI drawing, which is different from the HDMI output resolution. When the Framebuffer resolution differs from the HDMI output resolution, a corresponding scaling will be performed. You can set the Framebuffer resolution by configuring the following properties:

```
persist.sys.framebuffer.main=1920x1080
```

After 9.0, the property changes to:

```
persist.vendor.framebuffer.main=1920x1080
```

2.2.4 Resolution Filtering Configuration

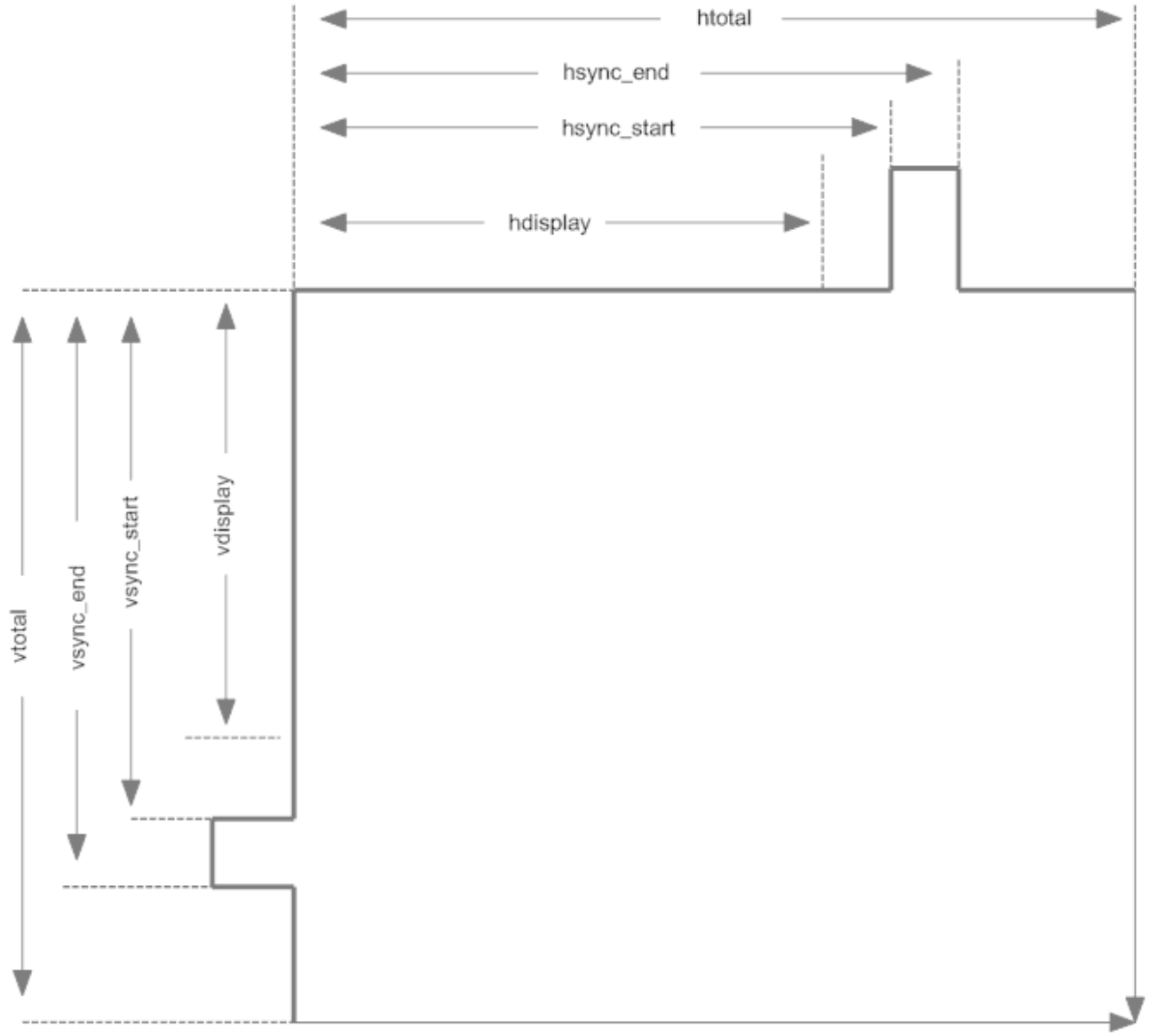
Due to the excessive number of resolutions initially obtained, some resolutions are not required by users. Therefore, filtering of resolutions has been implemented in the HWC module of the SDK. A whitelist approach is used to filter resolutions:

```
device/rockchip/common/resolution_white.xml
```

In HWC, the initial resolutions are filtered based on this configuration file before being passed to the upper layer. Each resolution block in this XML file defines a resolution that can pass the filter, with the detailed items defined as follows:

Item Definition	Description
clock	Pixel clock
hdisplay	Horizontal active pixels
hsync_start	Horizontal sync start pixel
hsync_end	Horizontal sync end pixel
htotal	Total horizontal pixels
hskew	Horizontal skew, usually 0
vdisplay	Vertical active lines
vsync_start	Vertical sync start line
vsync_end	Vertical sync end line
vtotal	Total vertical lines
vscan	Vertical scan signal, usually 0
vrefresh	Display device refresh rate
flags	Definitions of flags are as follows: DRM_MODE_FLAG_PHSYNC (1<<0) DRM_MODE_FLAG_NHSYNC (1<<1) DRM_MODE_FLAG_PVSYNC (1<<2) DRM_MODE_FLAG_NVSYNC (1<<3) DRM_MODE_FLAG_INTERLACE (1<<4)
vic	VIC value defined by HDMI standards, set to 0 if not defined in HDMI standards

For specific timing details, see the diagram below:



2.2.5 HDMI Configuration Options

The system's settings app can modify the current HDMI resolution and other properties from the UI.

To display the HDMI options in the settings, Android 7.X shows them by default; for Android 8.X and above, add the following configuration property to the product directory under the device:

```
BOARD_SHOW_HDMI_SETTING := true
```

The UI interface defaults to only displaying the secondary screen configuration. If you need to modify this, please change the following content in `HdmiSettings.java` within the `package/apps/Settings` code:

```
int value = SystemProperties.getInt("persist.hdmi.ui.state", ???);
```

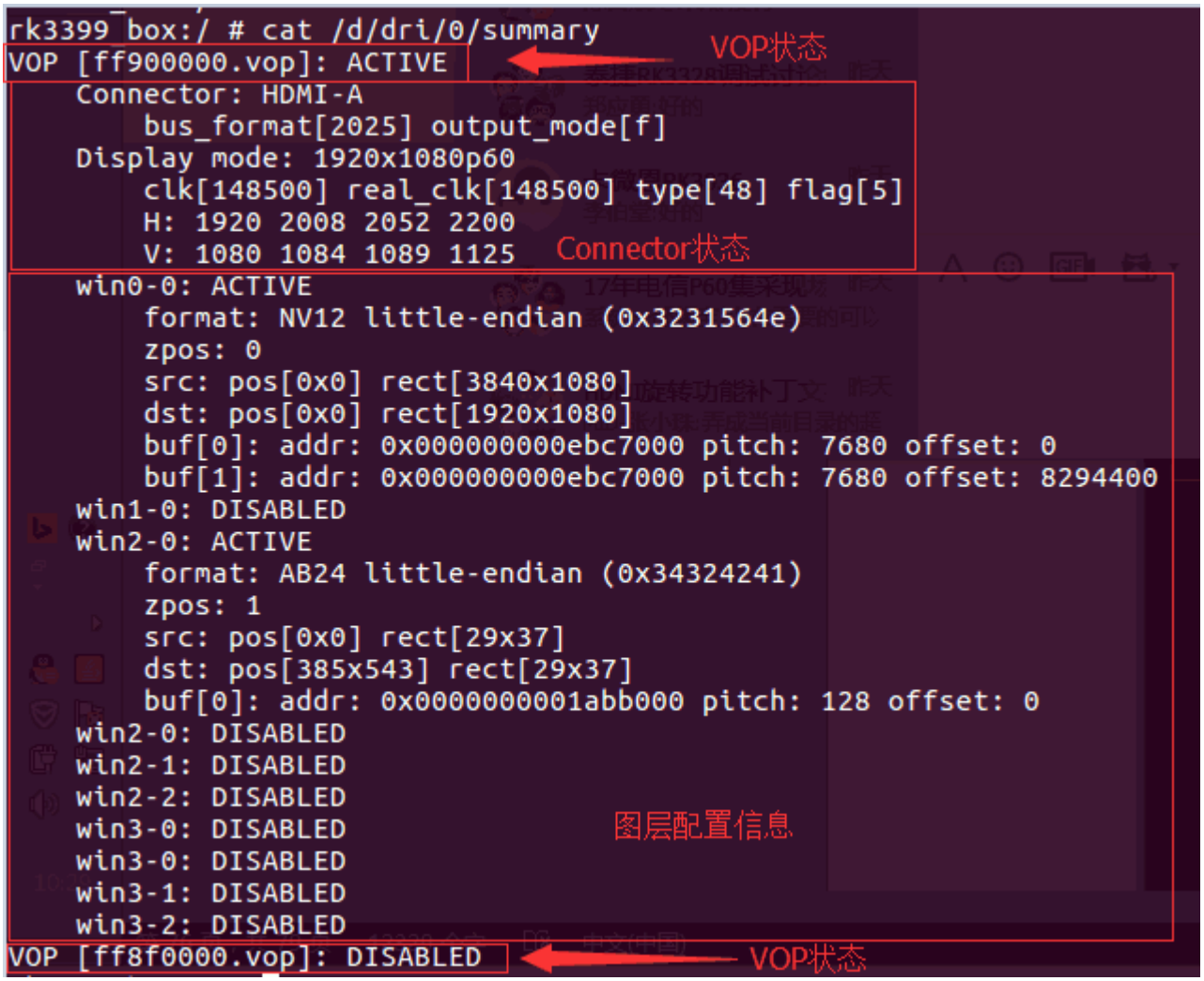
In the code, the value of `???` is 0: display secondary screen configuration UI; 1: display primary screen configuration UI; 2: display both primary and secondary screen UI configurations.

2.3 Common Debugging Methods

2.3.1 Checking VOP Status

Execute the following command to check the VOP status:

```
cat /sys/kernel/debug/dri/0/summary
```



The above command outputs the LOG when RK3399 is connected to HDMI, providing three types of information:

- VOP Status: VOPB is enabled, and VOPL is disabled.
- VOP Corresponding Connector Status: VOPB outputs a signal to HDMI, bus_format = 0x2025 indicates YUV444 8bit, output_mode = 0x0f indicates that the VOP output bus is ROCKCHIP_OUT_MODE_AAAA, outputting 1920x1080P60.
  Common bus_formats are defined by the kernel `uapi/linux/media-bus-format.h`:

```
#define MEDIA_BUS_FMT_RGB888_1X24            0x100a  //RGB888
#define MEDIA_BUS_FMT_RGB101010_1X30         0x1018  //RGB101010
#define MEDIA_BUS_FMT_YUV8_1X24              0x2025  //YUV444 8bit
#define MEDIA_BUS_FMT_YUV10_1X30             0x2016  //YUV444 10bit
#define MEDIA_BUS_FMT_UYYVYY8_0_5X24         0x2026  //YUV420 8bit
#define MEDIA_BUS_FMT_UYYVYY10_0_5X30        0x2027  //YUV420 10bit
```

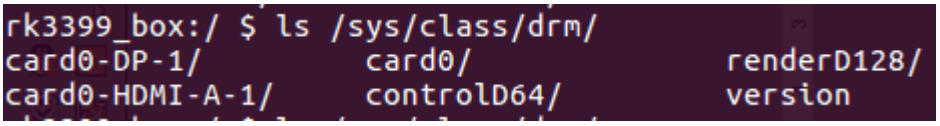
Common output_modes are defined by the kernel `drivers/gpu/drm/rockchip/rockchip_drm_vop.h`:

```
#define ROCKCHIP_OUT_MODE_P888    0
#define ROCKCHIP_OUT_MODE_P666    1
#define ROCKCHIP_OUT_MODE_P565    2
#define ROCKCHIP_OUT_MODE_S888    8
#define ROCKCHIP_OUT_MODE_S888_DUMMY    12
#define ROCKCHIP_OUT_MODE_YUV420    14
/* for use special outface */
#define ROCKCHIP_OUT_MODE_AAAA    15
```

- Layer Configuration Information: win0 and win2 are enabled, win2 buffer format is ARGB, buffer size is 29x37; target window is 29x37, window top-left corner coordinates (385,543). Win0 buffer format is NV12, size is 3840x2160; target window size is 1920x1080, window top-left corner coordinates (0,0).

2.3.2 Checking Connector Status

Under the `/sys/class/drm` directory, you can see the graphics cards registered by the driver. The following image shows the DRM directory structure of the RK3399 BOX platform, where `card0-HDMI-A-1` and `card0-DP-1` are registered as two display devices, representing HDMI and DP respectively.



Taking `card0-HDMI-A-1` as an example, the following files are present in its directory:

- Enabled: Enable status
- Status: Connection status
- Mode: Current output resolution
- Modes: List of supported resolutions by the connected device
- Audioformat: Audio formats supported by the connected device
- Edid: EDID of the connected device, which can be saved using the command `cat edid > /data/edid.bin`.

2.3.3 Check HDMI Operation Status

If the following commit is included, you can check the HDMI operation status:

```
commit eaca91814449199b1e6ad0b9fe0bba2215497c97
Author: Zheng Yang <zhengyang@rock-chips.com>
Date:   Mon Nov 27 16:56:21 2017 +0800

    drm: bridge: dw-hdmi: add hdmi status debugfs node
```

Use the following command to check the current HDMI output status:

```
cat /sys/kernel/debug/dw-hdmi/status
```

```
HDMI Output Status: PHY disabled
```

```
HDMI Output Status: PHY enabled
Pixel Clk: 148500000Hz          TMDS Clk: 148500000Hz
Color Format: YUV444            Color Depth: 8 bit
Colorimetry: ITU.BT709          EOTF: SDR
x0: 0                           y0: 0
x1: 0                           y1: 0
x2: 0                           y2: 0
white x: 0                      white y: 0
max lum: 0                      min lum: 0
max cll: 0                      max fall: 0
```

- HDMI Output Status indicates the current PHY status, which will only be printed after PHY is enabled.
- Pixel Clk represents the current output pixel clock.
- TMDS Clk represents the current output HDMI character rate.
- Color Format indicates the output color format, with values RGB, YUV444, YUV422, YUV420.
- Color Depth indicates the output color depth, with values 8bit, 10bit, 12bit, 16bit.
- Colorimetry indicates the output color standard, with values ITU.BT601, ITU.BT709, ITU.BT2020.
- EOTF indicates the output HDR electro-optical transfer function curve, with the following values:

EOTF	**Meaning**
Unsupported	HDMI does not support sending HDR information
Not Defined	Undefined
Off	Do not send HDR information
SDR	Use SDR curve
ST2084	Use ST2084 EOTF curve
HLG	Use HLG EOTF curve

- (x0, y0), (x1, y1), (x2, y2), (white x, white y), max lum, min lum, max cll, MaxFALL are static HDR metadata information, which only exists when the EOTF value is SDR, ST2084, HLG.

Execute the following command to view the HDMI controller registers:

```
cat /sys/kernel/debug/dw-hdmi/ctrl
```

You can modify the registers using commands, for example, to modify register 0x1000 to 0xF8, enter the command:

```
echo 1000 f8 > /sys/kernel/debug/dw-hdmi/ctrl
```

Platforms RK3288, RK3368, RK3399 can use the following command to view HDMI PHY registers:

```
cat /sys/kernel/debug/dw-hdmi/phy
```

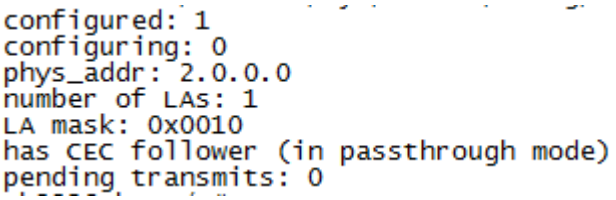
Modifying PHY registers is similar to the controller, for example, to modify register 0x06 to 0x8002, enter the command:

```
echo 06 8002 > /sys/kernel/debug/dw-hdmi/phy
```

2.3.4 Checking HDMI CEC Status

Execute the following command to check the HDMI CEC status:

```
cat /sys/kernel/debug/cec/cec0/status
```



The printed result is as shown in the figure above:

- **configured** indicates whether the CEC adapter has been configured, with 1 meaning configured and 0 meaning not yet configured.
- **configuring** indicates whether the CEC adapter is currently being configured, with 1 meaning it is in the process and 0 meaning it has either been completed or not yet started.
- **phys_addr** represents the physical address of CEC, which is f.f.f.f when the physical address has not been obtained.
- **number of LAs** indicates the number of logical addresses for this CEC device, with most devices having 1 and a few having 2.
- **LA mask** represents the current bound logical address, with specific values being (1 << bound logical address). For example: if the value is 0x0010, shifting right by 4 bits gives 1, indicating that the current logical address is 4. If the value is 0x0800, shifting right by 11 bits gives 1, indicating that the current logical address is 11. If no logical address has been bound, it will be 0x0000.
- **has CEC follower** indicates that the current received CEC messages are handed over to the upper user space for processing. In **passthrough mode**, the kernel does not process CEC core messages but instead reports all of them to the upper user space for processing.

- **pending transmits** indicates the number of CEC messages that are currently pending transmission.

2.3.5 Force Enable/Disable HDMI

Force Enable HDMI:

```
echo on > /sys/class/drm/card0-HDMI-A-1/status
```

Force Disable HDMI:

```
echo off > /sys/class/drm/card0-HDMI-A-1/status
```

Restore Hot-Plug Detection:

```
echo detect > /sys/class/drm/card0-HDMI-A-1/status
```

2.3.6 Command Line Resolution Configuration

In the Android system, resolution can be set using command-line property settings. Additionally, when users set the resolution in the Android settings, the corresponding property values are also set, as detailed in the following explanation.

2.3.6.1 Resolution Settings for Android 7.x & Android 8.x

Attribute	Description
persist.sys.resolution.main	Sets the main screen resolution, parameters are the timing sequence of this resolution, see section 2.2.4 for details.
persist.sys.resolution.aux	Sets the auxiliary screen resolution, parameters are the timing sequence of this resolution, see section 2.2.4 for details.
sys.display.timeline	Refreshes the display timeline, increment by one each time a new resolution is set.

By setting `persist.sys.resolution.main` and `persist.sys.resolution.aux` to adjust the main and auxiliary screen resolutions, update `sys.display.timeline` (increment by 1) and then perform operations such as moving the mouse to refresh the UI to make the new resolution take effect. Examples are as follows:

- Set 4k60:

  ```
  setprop persist.sys.resolution.main 3840x2160@60-3840-4016-4104-4400-2160-2168-2178-2250-5
  setprop sys.display.timeline 1
  ```

- Set 1080p60:

```
setprop persist.sys.resolution.main 1920x1080@60-1920-2008-2052-2200-1080-1084-1089-1125-5
setprop sys.display.timeline 2
```

- Set 720P60:

```
setprop persist.sys.resolution.main 1280x720@60.00-1280-1390-1430-1650-720-725-730-750-5
setprop sys.display.timeline 3
```

- Set 480P60:

```
setprop persist.sys.resolution.main 720x480@59.94-720-736-798-858-480-489-495-525-a
setprop sys.display.timeline 4
```

2.3.6.2 Resolution Settings for Android 9.0 and Above

There are differences in the properties used to configure screen resolution between new and old versions of Android 9.0, which need to be distinguished based on the HWC version. Newer Android 9.0 SDK and higher Android versions use new properties.

```
commit 6f772a162893dd0242b6644fa32166e1ac15b2c0
Author: libin <bin.li@rockchip.com>
Date:   Wed May 12 15:00:05 2021 +0800

    DrmConnector : Add UpdateDisplayMode function

    In order to adapt to the three-screen version.

    The hwc switching resolution will be in accordance with
    the following priority:
     1.persist.vendor.resolution.<Connector-type>-<Connector-ID>
     2.persist.vendor.resolution.main(aux)
     3.baseparameter(if exist)
     4.use "Auto"

    Version: 1.1.137

    Signed-off-by: libin <bin.li@rockchip.com>
    Change-Id: I73e216047ed5423929d7a091572d717c4ffdf50c
```

When the HWC code includes the above commit, the new properties are used:

Property	Description
persist.vendor.resolution.HDMI-A-0	Sets the resolution for HDMI0; for HDMI1, it would be HDMI-A-1. The parameter is the timing sequence of this resolution, see section 2.2.4 for details.
vendor.display.timeline	Refreshes the display timeline; increment by one each time a new resolution is set.

If the HWC does not contain the above properties, the old properties are used:

Property	Description
persist.vendor.resolution.main	Sets the main screen resolution; the parameter is the timing sequence of this resolution, see section 2.2.4 for details.
persist.vendor.resolution.aux	Sets the auxiliary screen resolution; the parameter is the timing sequence of this resolution, see section 2.2.4 for details.
vendor.display.timeline	Refreshes the display timeline; increment by one each time a new resolution is set.

By setting the main and auxiliary screen resolutions with `persist.vendor.resolution.main` and `persist.vendor.resolution.aux`, and updating `vendor.display.timeline` (increment by 1) after each setting, followed by moving the mouse or other UI update operations to make the new resolution take effect, examples are as follows:

- Setting 4k60:

```
setprop persist.vendor.resolution.main 3840x2160@60-3840-4016-4104-4400-2160-2168-2178-2250-5
setprop vendor.display.timeline 1
```

- Setting 1080p60:

```
setprop persist.vendor.resolution.main 1920x1080@60-1920-2008-2052-2200-1080-1084-1089-1125-5
setprop vendor.display.timeline 2
```

- Setting 720P60:

```
setprop persist.vendor.resolution.main 1280x720@60.00-1280-1390-1430-1650-720-725-730-750-5
setprop vendor.display.timeline 3
```

- Setting 480P60:

```
setprop persist.vendor.resolution.main 720x480@59.94-720-736-798-858-480-489-495-525-a
setprop vendor.display.timeline 4
```

2.3.7 Setting Colors via Command Line

In the Android system, colors can be set using command-line property settings. Additionally, when users set colors in Android's settings, the corresponding property values are also set. For more details, see the explanation below.

2.3.7.1 Android 7.x & Android 8.x Color Settings

Attribute	Description
persist.sys.color.main	Sets the main screen color. Parameters are: color format-bit depth For example, to set the color to RGB with a bit depth of 8 bits (24 bits), the parameter would be RGB-8bit Supported color formats include: RGB YCbCr444 YCbCr422 YCbCr420 Supported bit depths include: 8bit 10bit
persist.sys.color.aux	Sets the auxiliary screen color, with parameters the same as for the main screen.
sys.display.timeline	Refreshes the display timeline, incrementing by one each time a new resolution is set.

By setting `persist.sys.color.main` and `persist.sys.color.aux` for the main and auxiliary screen colors, respectively, and updating `sys.display.timeline` (incrementing by 1) after each setting, followed by moving the mouse or other UI update operations to make the new color take effect. Example as follows:

```
setprop persist.sys.color.main RGB-8bit
setprop sys.display.timeline 1
```

This sets the output color to RGB with a bit depth of 8 bits (RGB 24 bits).

2.3.7.2 Color Settings for Android 9.0 and Later versions

The color setting attributes used by Android versions differ from those described in 2.3.7, with new and old versions of Android using different color setting properties.

```
commit 6f772a162893dd0242b6644fa32166e1ac15b2c0
Author: libin <bin.li@rockchip.com>
Date:   Wed May 12 15:00:05 2021 +0800

    DrmConnector : Add UpdateDisplayMode function

    In order to adapt to the three-screen version.

    The hwc switching resolution will be in accordance with
    the following priority:
      1.persist.vendor.resolution.<Connector-type>-<Connector-ID>
```

```
    2.persist.vendor.resolution.main(aux)
    3.baseparameter(if exist)
    4.use "Auto"

  Version: 1.1.137

  Signed-off-by: libin <bin.li@rockchip.com>
  Change-Id: I73e216047ed5423929d7a091572d717c4ffdf50c
```

When the HWC code includes the above commit, the new property is used:

Property	Description
persist.vendor.color.HDMI-A-0	Sets the HDMI0 color; if setting HDMI1, it would be HDMI-A-1. Parameters: Color format-Color depth For example, to set the color to RGB, color depth 8 bit (24 bit), the parameter would be RGB-8bit Supported color formats include: RGB YCbCr444 YCbCr422 YCbCr420 Supported color depths include: 8bit 10bit
vendor.display.timeline	Refreshes the display timeline, incrementing with each new resolution setting.

If the HWC does not contain the above properties, the old version properties are used:

Property	Description
persist.vendor.color.main	Sets the main screen color, parameters: Color format-Color depth For example, to set the color to RGB, color depth 8 bit (24 bit), the parameter would be RGB-8bit Supported color formats include: RGB YCbCr444 YCbCr422 YCbCr420 Supported color depths include: 8bit 10bit
persist.vendor.color.aux	Sets the auxiliary screen color, parameters are the same as for the main screen.
vendor.display.timeline	Refreshes the display timeline, incrementing with each new resolution setting.

By setting `persist.vendor.color.main` and `persist.vendor.color.aux` to set the main and auxiliary screen colors, update `vendor.display.timeline` (increment by 1) after each setting, then perform operations such as moving the mouse to update the UI to make the new color take effect, example as follows:

```
setprop persist.vendor.color.main RGB-8bit
setprop vendor.display.timeline 1
```

This sets the output color to RGB, with a color depth of 8 bit (RGB 24 bit).

2.3.8 Setting Overscan

Due to differences among various televisions, the displayed image may exceed the screen boundaries or have black borders. In such cases, overscan can be adjusted to resize the image and correct these issues.

In the Android system, overscan can be set using command-line property settings. Additionally, users can also set overscan in the Android settings, and upon completion, the corresponding property values will be set as well. For details, see the explanation below.

2.3.8.1 Android 7.x & Android 8.x Overscan Settings

Attribute	Description
persist.sys.overscan.main	Sets the main screen overscan, property format: overscan left,top,right,bottom left, top, right, bottom correspond to the overscan values for the left, top, right, and bottom directions, respectively. The minimum value is 1, and the maximum value is defined by the property persist.sys.overscan.max. If persist.sys.overscan.max does not exist, the default value is 100.
persist.sys.overscan.aux	Sets the auxiliary screen overscan, parameters are the same as for the main screen.

Example:

```
setprop persist.sys.overscan.main "overscan 70,70,70,70"
```

Sets the overscan for all four directions to 70.

2.3.8.2 Overscan Settings for Android 9.0 and Later versions

Refer to section 2.3.8 for instructions, and select the properties to set based on the Android version.

New version properties:

Property	Description
persist.vendor.overscan.HDMI-A-0	Sets the overscan for HDMI0; if setting HDMI1, it would be HDMI-A-1. Property format: overscan left,top,right,bottom left, top, right, bottom represent the overscan values for the left, top, right, and bottom directions, respectively. The minimum value is 1, and the maximum value is defined by the property persist.vendor.overscan.max. If persist.vendor.overscan.max does not exist, the default is 100.

Old version properties:

Property	Description
persist.vendor.overscan.main	Sets the overscan for the main screen. Property format: overscan left,top,right,bottom left, top, right, bottom represent the overscan values for the left, top, right, and bottom directions, respectively. The minimum value is 1, and the maximum value is defined by the property persist.vendor.overscan.max. If persist.vendor.overscan.max does not exist, the default is 100.
persist.vendor.overscan.aux	Sets the overscan for the auxiliary screen, with the same parameters as the main screen.

Example:

```
setprop persist.vendor.overscan.main "overscan 70,70,70,70"
```

Sets the overscan for all four directions to 70.

2.3.9 Setting Brightness, Contrast, Saturation, and Hue

In the Android system, these parameters can be set using command-line property settings. Additionally, users can also set these parameters in the Android settings. Once the settings are completed, the corresponding property values will also be set, as detailed in the following explanation.

2.3.9.1 Android 7.x & Android 8.x Brightness, Contrast, Saturation, and Hue Settings

BCSH	Value Range	Description
Brightness	Integer, 0 - 100, default value 50	persist.sys.brightness.main persist.sys.brightness.aux
Contrast	Integer, 0 - 100, default value 50	persist.sys.contrast.main persist.sys.contrast.aux
Saturation	Integer, 0 - 100, default value 50	persist.sys.saturation.main persist.sys.saturation.aux
Hue	Integer, 0 - 100, default value 50	persist.sys.hue.main persist.sys.hue.aux

For example:

```
setprop persist.sys.brightness.main 70
setprop sys.display.timeline 1
```

By setting `persist.sys.brightness.main` to adjust the main screen brightness, after each setting, update `sys.display.timeline` (increment by 1) and then perform operations such as moving the mouse to refresh the UI to make the new brightness take effect.

2.3.9.2 Brightness, Contrast, Saturation, and Hue Settings for Android 9.0 and Above

Refer to section 2.3.9 for instructions on selecting properties based on the Android version for configuration.

New version properties:

BCSH	Value Range	Description
Brightness	Integer, 0 - 100, default value 50	persist.vendor.brightness.HDMI-A-0
Contrast	Integer, 0 - 100, default value 50	persist.vendor.contrast.HDMI-A-0
Saturation	Integer, 0 - 100, default value 50	persist.vendor.saturation.HDMI-A-0
Hue	Integer, 0 - 100, default value 50	persist.vendor.hue.HDMI-A-0

Old version properties:

BCSH	Value Range	Description
Brightness	Integer, 0 - 100, default value 50	persist.vendor.brightness.main persist.vendor.brightness.aux
Contrast	Integer, 0 - 100, default value 50	persist.vendor.contrast.main persist.vendor.contrast.aux
Saturation	Integer, 0 - 100, default value 50	persist.vendor.saturation.main persist.vendor.saturation.aux
Hue	Integer, 0 - 100, default value 50	persist.vendor.hue.main persist.vendor.hue.aux

Example:

```
setprop persist.vendor.brightness.main 70
setprop vendor.display.timeline 1
```

Set the main screen brightness using `persist.vendor.brightness.main`, and after each setting, update `vendor.display.timeline` (increment by 1) and then perform operations such as moving the mouse to refresh the UI to make the new brightness take effect.

2.4 Troubleshooting Common Issues

2.4.1 Inserting or switching resolutions, TV prompts no signal or format not supported or unstable image, intermittent, or with a large number of colored bright spots, bright lines

1. Confirm the current HDMI resolution, command see 2.3.2.
2. Reduce the HDMI resolution to see if the TV can display normally, command see 2.3.6.
3. Replace with a better HDMI cable to see if the TV can display normally.
4. If steps 2 and 3 can restore the image, it is generally related to the compatibility of the HDMI physical signal, and it is necessary to check the hardware and test the HDMI signal for further analysis.
5. If the HDMI signal is not up to standard, signal adjustment can be made by adjusting the configuration of HDMI PHY, reference 2.1.8.

2.4.2 Video Playback Results in TV Signal Loss or Unsupported Format Notification

Inspect the kernel code `dts` for the presence of DDR frequency modulation functionality specific to video; if found, set `auto-freq-en = <0>;` to disable the automatic frequency modulation feature.

```
dmc: dmc {
        compatible = "rockchip,rk3328-dmc";
        devfreq-events = <&dfi>;
        clocks = <&cru SCLK_DDRCLK>;
        clock-names = "dmc_clk";
        operating-points-v2 = <&dmc_opp_table>;
        ddr_timing = <&ddr_timing>;
        upthreshold = <40>;
        downdifferential = <20>;
        system-status-freq = <
                /*system status         freq(KHz)*/
                SYS_STATUS_NORMAL       786000
                SYS_STATUS_REBOOT       786000
                SYS_STATUS_SUSPEND      786000
                SYS_STATUS_VIDEO_1080P  786000
                SYS_STATUS_VIDEO_4K     786000
                SYS_STATUS_VIDEO_4K_10B 933000
                SYS_STATUS_PERFORMANCE  933000
                SYS_STATUS_BOOST        933000
        >;
        auto-min-freq = <786000>;
        auto-freq-en = <0>;
        #cooling-cells = <2>;
        status = "disabled";
};
```

2.4.3 Some TVs show no signal, black screen, or garbled screen

1. Confirm the current HDMI resolution, command see 2.3.2.
2. Reduce the HDMI resolution to see if the TV can display normally, command see 2.3.6.
3. Replace with a good HDMI cable to see if the TV can display normally.
4. If steps 2 and 3 can restore the picture, it is generally related to the compatibility of the HDMI physical signal, and hardware needs to be checked to further analyze the HDMI signal.
5. If the HDMI signal does not meet the standard, the signal can be adjusted by adjusting the configuration of HDMI PHY, refer to 2.1.8.
6. If the HDMI signal is tested to meet the standard, you can try to modify the following register values:

   kernel/drivers/gpu/drm/bridge/synopsys/dw-hdmi.c

   ```
   /* HDMI Initialization Step B.4 */
   static void dw_hdmi_enable_video_path(struct dw_hdmi *hdmi)
   {
      /* control period minimum duration */
      hdmi_writeb(hdmi, 12, HDMI_FC_CTRLDUR);
   ```

   Gradually increase the value of HDMI_FC_CTRLDUR (maximum 223) to see if the display can return to normal.

2.4.4 How to Set Default Resolution When Reading EDID Fails

```
commit 727e0fe68d8f422698f4e257cb7c04f90b8692c0
Author: xuhuicong <xhc@rock-chips.com>
Date:   Tue Sep 26 17:32:56 2017 +0800
drm/edid: output common tv resolution and hdmi mode if no read the correct edid
Change-Id: Ib7379340e8c1d59382553d21b60165fe5fb371e8
Signed-off-by: xuhuicong <xhc@rock-chips.com>
```

Based on the above commit, modify the value of `def_modes`, which corresponds to the VIC values, such as the 4 corresponding to the 720P60 resolution in the following code.

kernel/drivers/gpu/drm/bridge/synopsys/dw-hdmi.c

```
static int dw_hdmi_connector_get_modes(struct drm_connector *connector)
{
   struct dw_hdmi *hdmi = container_of(connector, struct dw_hdmi,
                       connector);
   struct edid *edid;
   struct drm_display_mode *mode;
   const u8 def_modes[6] = {4, 16, 31, 19, 17, 2};
   struct drm_display_info *info = &connector->display_info;
```

2.4.5 Force Output of Specified Resolution

When it is necessary to ignore the limitations of EDID and force the output of a specific resolution, confirm whether the current code for RK3288/RK3368/RK3399/RK356X includes the following commit:

```
commit b318f175080ca98173d75fcf436beeee64092303
Author: Algea Cao <algea.cao@rock-chips.com>
Date:   Fri Nov 17 09:55:52 2023 +0800

    drm/bridge: Synopsys: Support hdmi force output

    Support hdmi output specific resolution and color format
    regardless of whether hdmi is connected.

    Signed-off-by: Algea Cao <algea.cao@rock-chips.com>
    Change-Id: I228a74d128aa818166f589798897729473d97610
```

For RK3588/RK3576, confirm whether the current code includes the following commit:

```
commit ed5631fa515a297b45cef5a1330b516f56e113dc
Author: Algea Cao <algea.cao@rock-chips.com>
Date:   Thu Dec 28 10:16:17 2023 +0800

    drm/bridge: dw-hdmi-qp: Support hdmi force output

    Support hdmi output specific resolution and color format
    regardless of whether hdmi is connected.

    Signed-off-by: Algea Cao <algea.cao@rock-chips.com>
    Change-Id: I3f18aeb04427846e06b6a4397a4c6df77bbbcab2
```

If these commits are included, you can directly configure the desired resolution and color format in the DTS:

```
&hdmi {
        status = "okay";
        force-output;
        force-bus-format = <MEDIA_BUS_FMT_RGB888_1X24>;
        force_timing {
                clock-frequency = <594000000>;
                hactive = <3840>;
                vactive = <2160>;
                hback-porch = <296>;
                hfront-porch = <176>;
                vback-porch = <72>;
                vfront-porch = <8>;
                hsync-len = <88>;
                vsync-len = <10>;
                hsync-active = <1>;
                vsync-active = <1>;
                de-active = <0>;
                pixelclk-active = <0>;
```

```
    };
};
```

1. `force-output`: indicates the FLAG for forcing HDMI resolution output.
2. `force-bus-format`: forces the output of a specified color format.
3. `force_timing`: forces the output of a specified resolution timing.

If the current code does not include the above commits, please make the following modifications:

```
diff --git a/drivers/gpu/drm/bridge/synopsys/dw-hdmi.c b/drivers/gpu/drm/bridge/synopsys/dw-hdmi.c
index 5ed021b..9a7f18c 100644
--- a/drivers/gpu/drm/bridge/synopsys/dw-hdmi.c
+++ b/drivers/gpu/drm/bridge/synopsys/dw-hdmi.c
@@ -2497,7 +2497,7 @@ static int dw_hdmi_connector_get_modes(struct drm_connector *connector)
                                           connector);
        struct edid *edid;
        struct drm_display_mode *mode;
-       const u8 def_modes[6] = {4, 16, 31, 19, 17, 2};
+       const u8 def_modes[6] = {107, 16, 31, 19, 17, 2};
        struct hdr_static_metadata *metedata =
                        &connector->display_info.hdmi.hdr_panel_metadata;
        int i, ret = 0;
@@ -2506,6 +2506,7 @@ static int dw_hdmi_connector_get_modes(struct drm_connector *connector)
                return 0;

        edid = drm_get_edid(connector, hdmi->ddc);
+       edid = NULL;
        if (edid) {
                dev_dbg(hdmi->dev, "got edid: width[%d] x height[%d]\n",
                        edid->width_cm, edid->height_cm);
diff --git a/drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c b/drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c
index bdc96cd4..6653dfa 100644
--- a/drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c
+++ b/drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c
@@ -488,10 +488,12 @@ dw_hdmi_rockchip_mode_valid(struct drm_connector *connector,
         * If sink max TMDS clock < 340MHz, we should check the mode pixel
         * clock > 340MHz is YCbCr420 or not.
         */
+#if 0
        if (mode->clock > 340000 &&
            connector->display_info.max_tmds_clock < 340000 &&
            !drm_mode_is_420(&connector->display_info, mode))
                return MODE_BAD;
+#endif

        if (!encoder) {
                const struct drm_connector_helper_funcs *funcs;
```

1. Change the first value of the `def_mode` array to the VIC corresponding to the desired resolution.
2. `edid = NULL;` forces the process to fail when reading EDID, regardless of whether EDID is read or not, and displays according to the resolution in def_modes.
3. If you need to force display a 4K resolution, you also need to comment out the code in the above image to remove the restriction on 4K resolution.

It should be noted that when forcing the output of HDMI 2.0 (4K30 and above) and higher resolutions, it is necessary to confirm whether the SINK end requires SCDC communication. If SCDC communication is required, it is essential to ensure that the HDMI DDC function is working properly. Also, HDMI hot-plugging is not supported during forced output.

2.4.6 Recovery HDMI Display Not Available

Recovery does not support dual displays nor hot-plugging. If you need to display from HDMI, apply the following modifications and then boot with HDMI connected.

```
diff --git a/minui/graphics_drm.cpp b/minui/graphics_drm.cpp
index 03e33b7..f4d5fc7 100644
--- a/minui/graphics_drm.cpp
+++ b/minui/graphics_drm.cpp
@@ -295,6 +295,7 @@ static drmModeConnector *find_main_monitor(int fd, drmModeRes *resources,
     int modes;
     /* Look for LVDS/eDP/DSI connectors. Those are the main screens. */
     unsigned kConnectorPriority[] = {
+        DRM_MODE_CONNECTOR_HDMIA,
         DRM_MODE_CONNECTOR_LVDS,
         DRM_MODE_CONNECTOR_eDP,
         DRM_MODE_CONNECTOR_DSI,
```

2.4.7 Unable to Set HDMI Resolution in settings

1. Please confirm whether the primary and secondary screen configurations in section 2.2.1 are correct, and the settings in section 2.2.5 are correct.
2. Please confirm whether the property configurations in section 2.3.6 are correct.
3. If it is an Android 9.X or higher system, the RkOutputManager service needs to be enabled. The RK3399 code must be updated to the following commit point.

```
commit 3550c7dd16aea55c67b58e45d42301d94216f665
Author: wh <wanghang@rock-chips.com>
Date:   Wed Apr 3 15:56:06 2019 +0800

    rk3399: mid add hdmi config

    Change-Id: I0705b537f9fe110bb06ae6390caaa8886f8d36a6
    Signed-off-by: wh <wanghang@rock-chips.com>

wh@ubuntu:~/3399_9.0/device/rockchip/rk3399$
```

4. For other platforms on 9.0, the corresponding patches need to be applied. The patch is to execute the `get_build_var DEVICE_MANIFEST_FILE` command after running the source and lunch-related commands in the current project, which will print the manifest file in use, for example, the output is: `device/rockchip/common/manifest.xml`, then add the following code to the manifest file:

```
<hal format="hidl">
    <name>rockchip.hardware.outputmanager</name>
    <transport>hwbinder</transport>
    <version>1.0</version>
    <interface>
        <name>IRkOutputManager</name>
        <instance>default</instance>
    </interface>
</hal>
```

2.4.8 DDR Bandwidth Insufficiency Issues

If flickering or green lines appear at 4K high resolution, confirm whether the kernel log has the following printout:

```
[drm:vop_isr] ERROR POST_BUF_EMPTY irq err
```

If the above printout is present, it indicates a DDR bandwidth insufficiency issue. Please refer to section 9.7 of the document **Rockchip_RK3399_Developer_Guide_Android7.1_Software_CN&EN.pdf** for resolution.

2.4.9 4K UI Related Issues

1. Is it necessary to have a 4K UI?

   The 4K UI consumes more system resources and can support up to approximately 4K25Hz, which is not recommended. If you only want to play 4K videos or view 4K images, it is not necessary to configure a 4K UI; the system's default video player and image browser can support these functions.

2. How to configure a 4K UI?

   Please refer to section 2.2.3 to configure the Framebuffer resolution to 4K.

3. After configuring to 4K UI, if there is a flickering screen or DDR bandwidth issue, please refer to section 2.4.8 for handling.

2.4.10 Setting HDMI Resolution List Missing 4K Resolution

1. Confirm whether the TV supports 4K resolution.
2. Execute the following command to confirm if the kernel's HDMI resolution list includes 4K resolution.

   ```
   cat /sys/class/drm/card0-HDMI-A-1/modes
   ```

3. If the above list does not include 4K resolution, for dual VOP platforms (RK3288, RK3399), confirm whether HDMI is bound to VOPB. Or check if the TV's 4K-50/60Hz does not support YUV420, as the current platform does not support such a high resolution (refer to Chapter 1 table for the highest supported HDMI resolution by the platform).
4. If the kernel's HDMI resolution list includes 4K resolution, but the setting's resolution list does not include this resolution, confirm whether the whitelist includes this resolution (refer to Section 2.2.4).

2.4.11 RK3588 Configuration: HDMI Resolution List Missing 8K Resolution

1. Verify if the TV settings have selected HDMI 2.1 mode or game mode, as most 8K TVs require manual configuration to display 8K resolution.
2. Confirm whether VOP ACLK is set to 800 MHz, refer to section 2.1.4.

2.4.12 RK3588/RK3576 HDMI 8K/4K120 and Other Resolutions Screen Flicker, Display Anomalies

1. Confirm whether the HDMI cable used is an HDMI 2.1 cable. HDMI 2.1 resolution must use an HDMI 2.1 cable.
2. Provide HDMI schematics / PCB diagrams for RK hardware review.
3. Contact an external lab to conduct SI testing or apply to the business for SI testing by the RK hardware lab.

2.4.13 HDMI Certification Application Form Filling

To certify a device with HDMI, one typically obtains a certification application form from the certification authority, which is usually in Excel format.

First, pay attention to the pagination tabs at the bottom of the form, as shown in the following example:



Certification for various HDMI functions requires filling out the content in each pagination. If the device does not support certain functions, the corresponding pagination does not need to be filled out. Common pagination descriptions are as follows:

- General: Most application forms have similar pages and must be filled out. They usually contain basic information about the device's HDMI, such as how many HDMI IN ports the device has, how many HDMI OUT ports, whether the device supports HDCP, CEC, and other functions. The options on this pagination often determine whether subsequent paginations need to be filled out. For example, if HDMI_input_count is filled with 0, it indicates that the device does not support HDMI IN, and the Sink CDF pagination does not need to be filled out.

Source/Sink/Repeater Characteristics

Must be filled out for any Source, Sink or Repeater. Do not fill out if Cable.

Product Category

Field Name	*Field Definition*	*Choices*	*Value*
HDMI_output_count	How many HDMI output ports are on the product?	0…X	1
HDMI_input_count	How many HDMI input ports are on the product?	0…X	0
HDCP_Supported	Is HDCP supported on the product?	Y/N	N

CEC Characteristics

Field Name	*Field Definition*	*Choices*	*Value*
CEC_protocol	Is CEC protocol supported? If this field is "Y", then CEC CDF shall be submitted.	Y/N	N
Independent_CEC	Are the CEC signals on input connectors independent? (Meaning: no physical connection between inputs and the product has a logical address of 0 for all inputs). [Note: If the product has no HDMI inputs, answer "N".]	Y/N	N
CEC_root_device	Does the product act as a CEC root device? (Meaning: the product is a Sink or Repeater and the product's Physical Address is 0.0.0.0 and the product's EDID(s) [if present] contain Source Physical Address of P.0.0.0). [Note: If the product has no HDMI inputs, answer "N".]	Y/N	N

HEAC Characteristics

Field Name	*Field Definition*	*Choices*	*Value*
HEC_supported	Is HEC supported? If this field is "Y", then HEAC CDF shall be submitted.	Y/N	N
ARC_supported	Is ARC supported? If this field is "Y", then HEAC CDF shall be submitted.	Y/N	N

- Source CDF: If the device includes HDMI OUT ports (the number of HDMI OUT ports supported must be filled out in the General pagination), this pagination needs to be filled out, which generally requires filling out the resolutions supported by HDMI OUT, the color formats supported, etc.

Which HDMI output ports are covered by this section? port1 (type in port names/numbers)

Field Name	*Field Definition*	*Choices*	*Value*
Electrical			
Source_DDC_cap_power_on	Should the DDC capacitance be measured with the product powered on? (Note: HPD will be false during measurement.)	Y/N	N
Video			
Source_HDMI_YCbCr	Will the product transmit an HDMI video signal using YCbCr (4:4:4 or 4:2:2) pixel encoding under some conditions (user selection, EDID indication etc.)?	Y/N	N
Source_AVI_Required	Is the product ever required to transmit an AVI InfoFrame	Y/N	Y
Source_AVI_Supported	Does the product support the transmission of the AVI InfoFrame under some conditions?	Y/N	Y
Source_AVI_Info_Available	Is any of the following information available and valid at the product?: Active Format Aspect Ratio, bar widths, overscan vs. underscan, non-uniform picture scaling, or the colorimetry of the video.	Y/N	Y
Source_Alt_Colorimetry	Will the product ever transmit video using a non-default (i.e. alternate) colorimetry under some condition? (e.g. using BT.709 for 480p or BT.601 for 1080i)	Y/N	N
Source_xvYCC	Is the product capable of transmitting video using xvYCC colorimetry?	Y/N	N
Source_AR_Converter	Does the product have the ability to convert between aspect ratios of 4:3 and 16:9 (and vice versa)?	Y/N	N

- Sink CDF: If the device includes HDMI IN ports, this pagination needs to be filled out, which generally

requires filling out the resolutions supported by HDMI IN, the color formats supported, and EDID-related information, etc.

- Repeater CDF: If the device acts as an HDMI Repeater, this pagination needs to be filled out. Rockchip solutions generally do not include this product form.

Each pagination generally includes several tables that need to be filled out, with the filling method being consistent with ordinary Excel tables. Taking the Video table under the Source CDF pagination as an example:

Video			
Source_HDMI_YCbCr	Will the product transmit an HDMI video signal using YCbCr (4:4:4 or 4:2:2) pixel encoding under some conditions (user selection, EDID indication etc.)?	Y/N	N
Source_AVI_Required	Is the product ever required to transmit an AVI InfoFrame	Y/N	Y N
Source_AVI_Supported	Does the product support the transmission of the AVI InfoFrame under some conditions?	Y/N	Y
Source_AVI_Info_Available	Is any of the following information available and valid at the product?: Active Format Aspect Ratio, bar widths, overscan vs. underscan, non-uniform picture scaling, or the colorimetry of the video.	Y/N	Y
Source_Alt_Colorimetry	Will the product ever transmit video using a non-default (i.e. alternate) colorimetry under some condition? (e.g. using BT.709 for 480p or BT.601 for 1080i)	Y/N	N
Source_xvYCC	Is the product capable of transmitting video using xvYCC colorimetry?	Y/N	N
Source_AR_Converter	Does the product have the ability to convert between aspect ratios of 4:3 and 16:9 (and vice versa)?	Y/N	N
Source_60Hz	Does the product output standard, enhanced or high-definition 60Hz video formats on any video output in addition to 640x480p @ 60Hz?	Y/N	Y
Source_50Hz	Does the product output standard, enhanced or high-definition 50Hz video formats on any video output?	Y/N	Y
Source_Above_165	Does the product support any video format/color mode with a TMDS clock frequency above 165MHz?	Y/N	N
Source_Deep_Color	Does the product support any Deep Color modes?	Y/N	N

For the options in the table, fill them out one by one according to the actual situation of the device. As shown in the Source_HDMI_YCbCr item in the figure above, the first column of the table is the item name, the second column is the item description, the third column is the optional value, and the fourth column is the value that the applicant needs to fill in. According to the description, this item indicates whether HDMI supports outputting the YCbCr color format. If this function is supported, select Y in the drop-down list on the right; otherwise, select N.

Since the application form usually has many items, they will not be explained one by one here. If there are any questions about the filling of any item, please raise them on redmine.